

LING2136 / 4136

**Advanced Statistical Methods for
Language Students**

Session-07: MCMC sampling

Lecturer: Timo Roettger

The background of the slide is a black surface with a complex network of white, thin, and jagged lines radiating from a central point, resembling a shattered glass or a cracked screen. The cracks are most dense in the center and become sparser towards the edges.

**Calculating posterior
distribution for our
models is computationally
intractable**

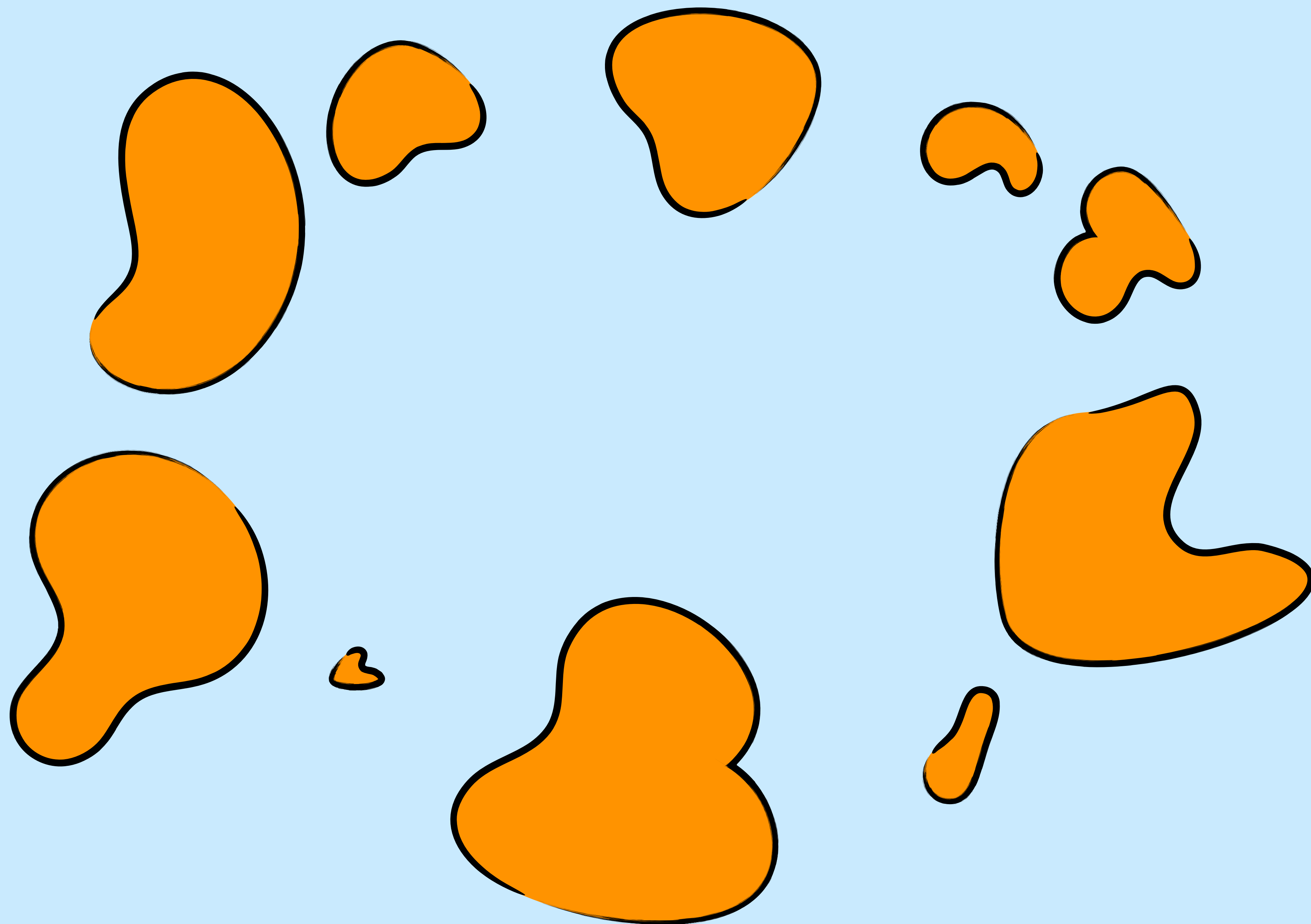


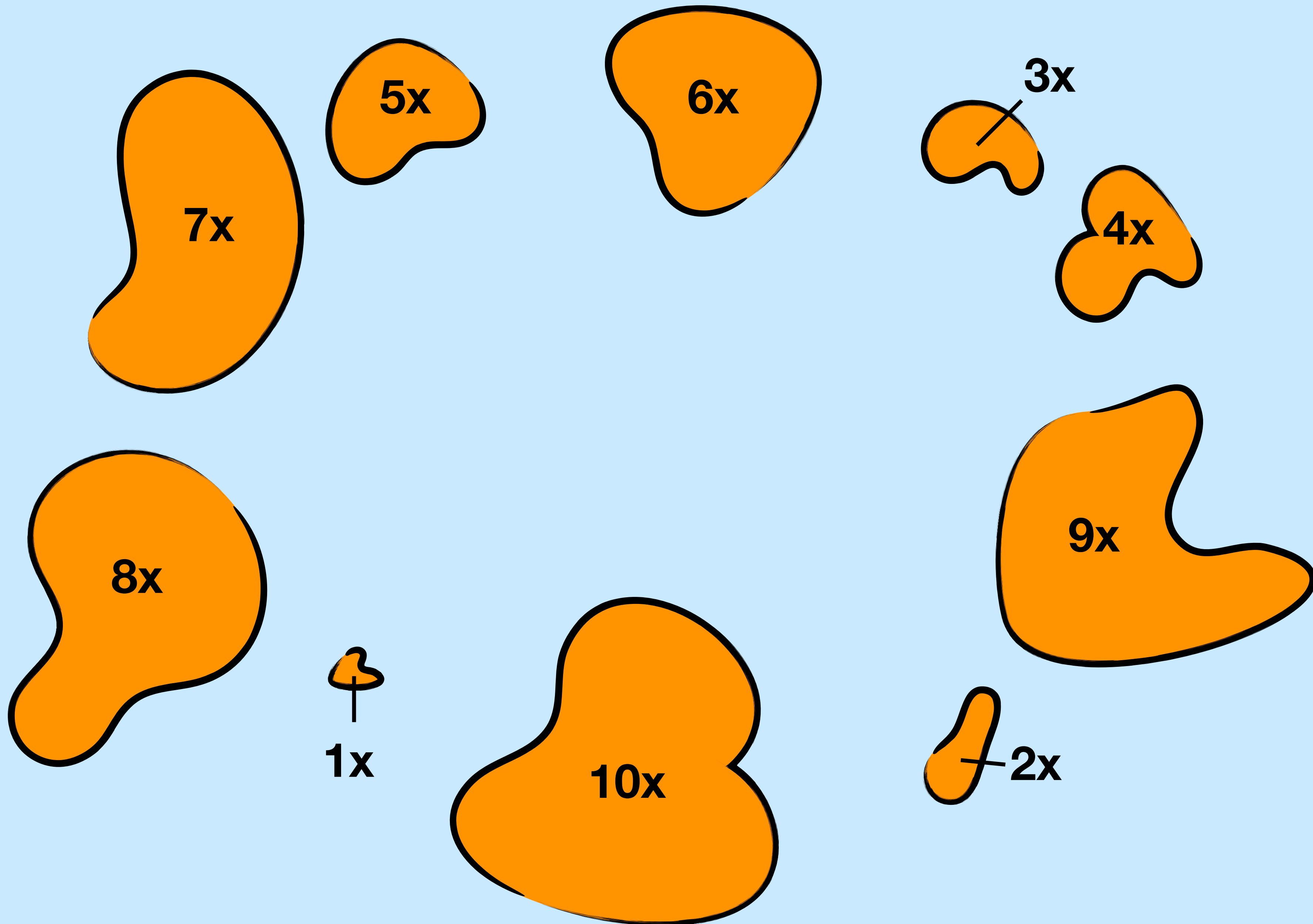
Markov Chain

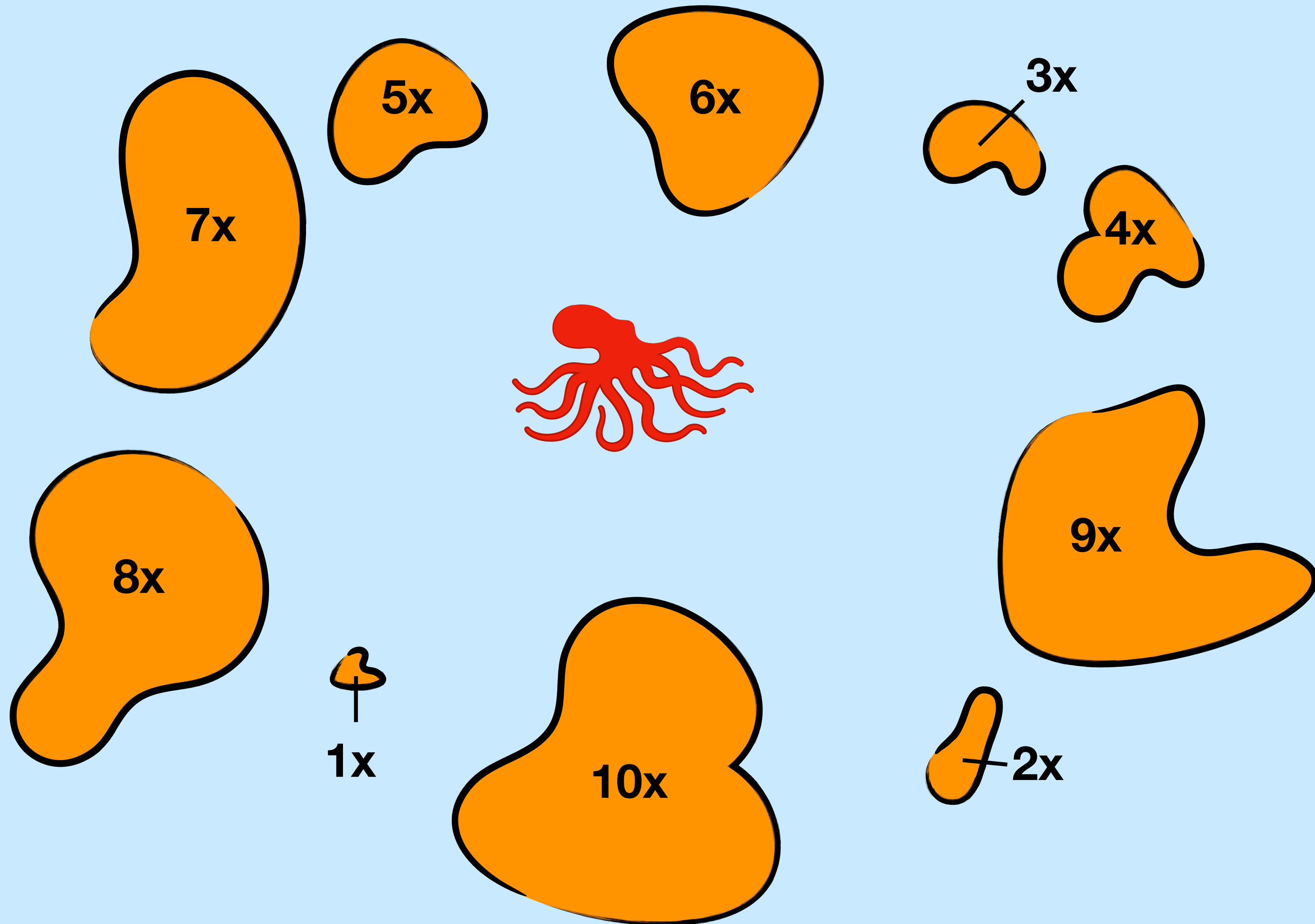
a sequence of samples
(chain) where each sample
depends on the previous
one

simulation using
random sampling

Monte Carlo

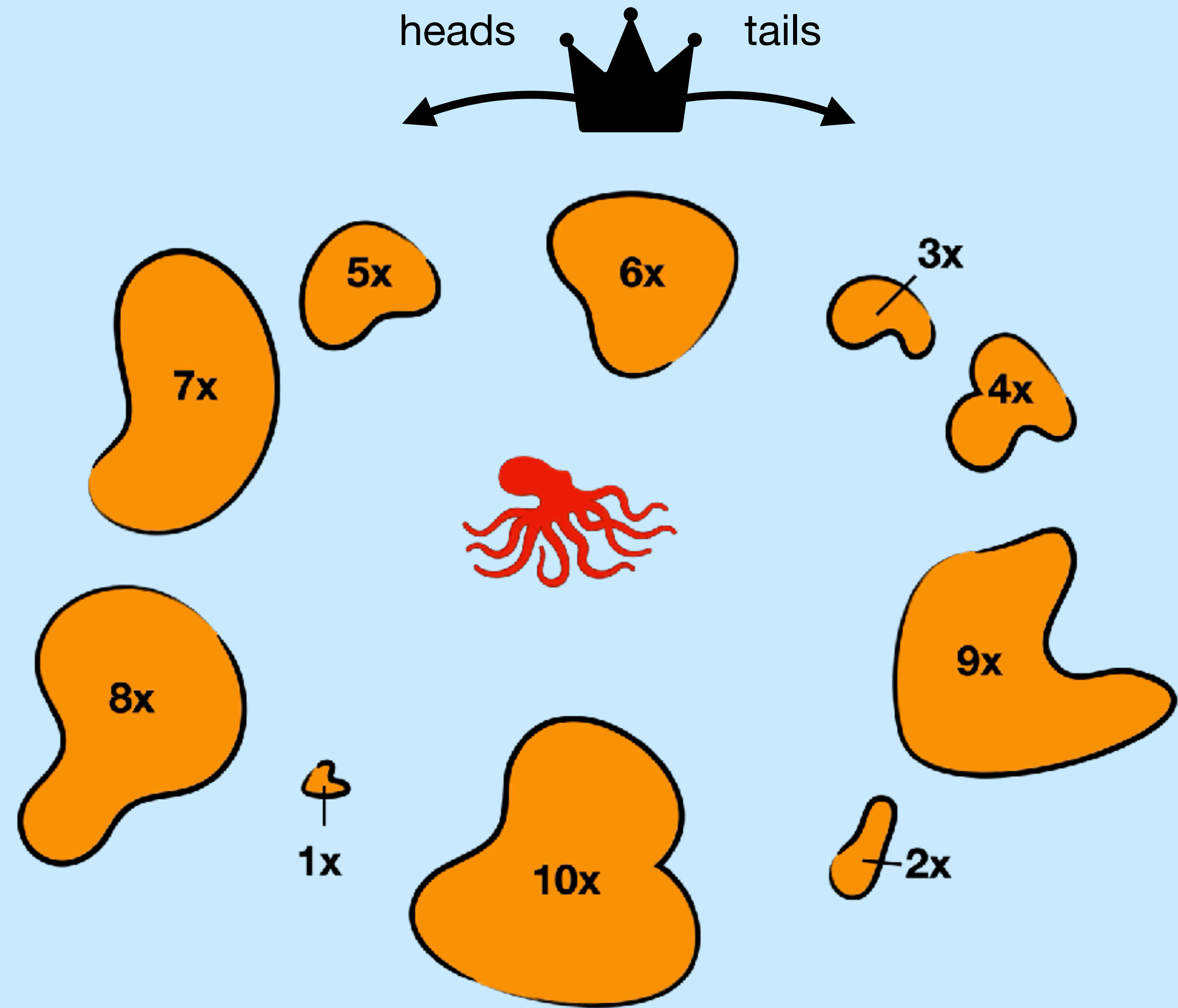






1. stay put or move to adjacent island by coin flip

2. if heads, considers clockwise movement, if tails, considers anti-clockwise



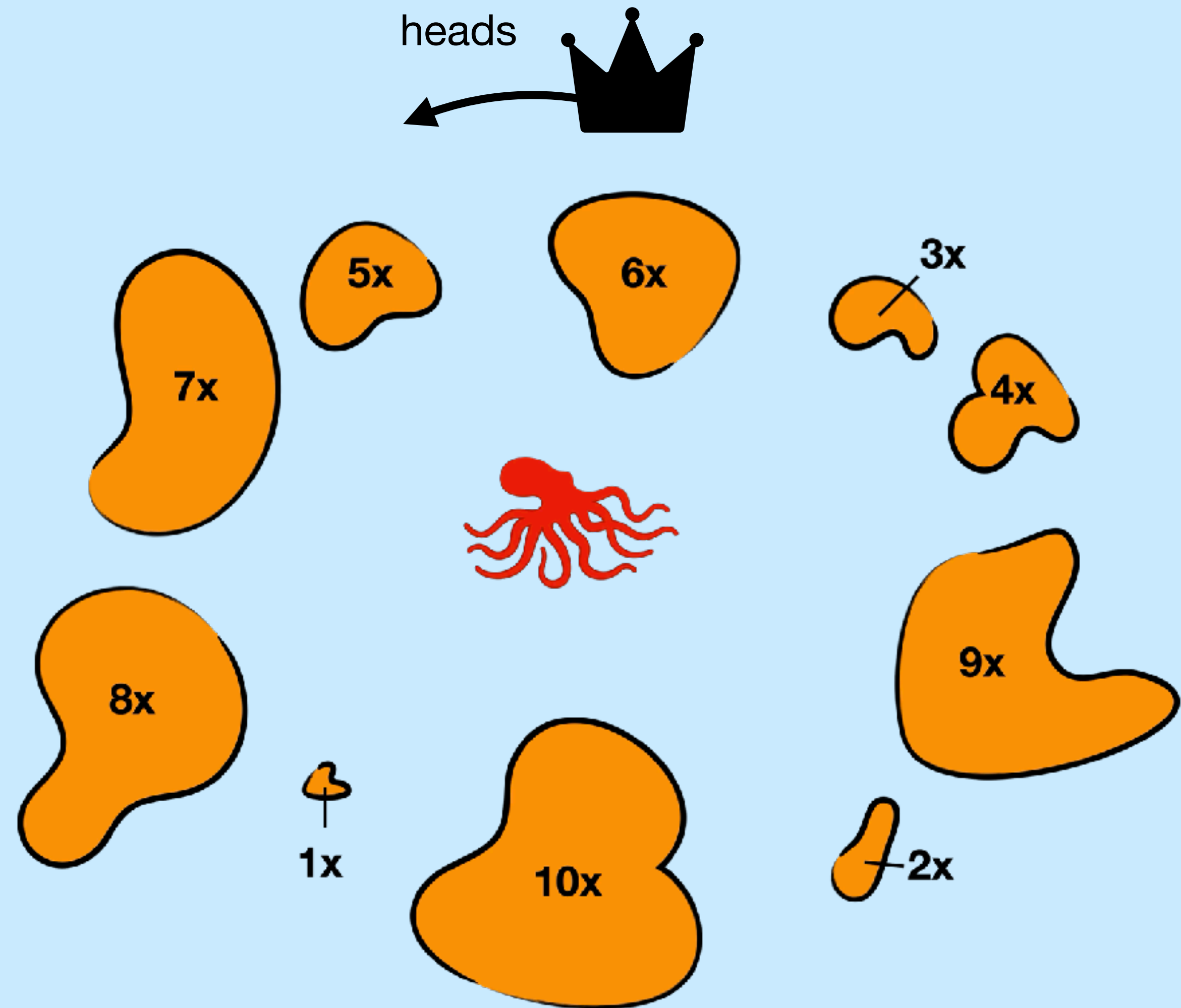
1. stay put or move to adjacent island by coin flip

2. if heads, considers clockwise movement, if tails, considers anti-clockwise

3. population size proposal
island = shells; population size
current island = stones

proposal island 

current island 



1. stay put or move to adjacent island by coin flip

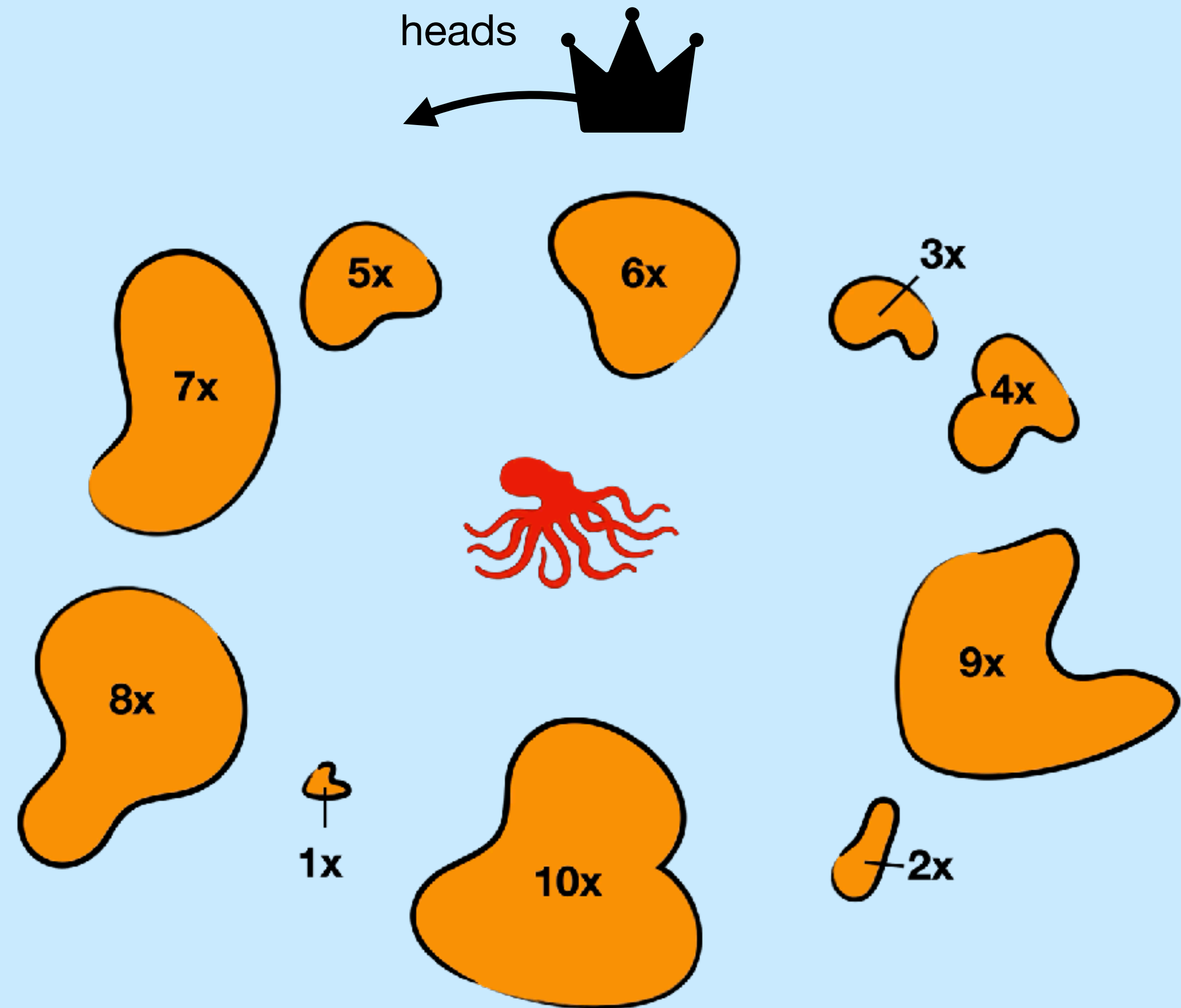
2. if heads, considers clockwise movement, if tails, considers anti-clockwise

3. population size proposal
island = shells; population size
current island = stones

4. if 🐚 > 🪨 move to proposal island;
if 🪨 > 🐚 discard # 🪨 = 🐚
place remaining stones and shells in a bag and randomly pull one out

proposal island 🐚🐚🐚🐚🐚

current island 🪨🪨🪨🪨🪨🪨



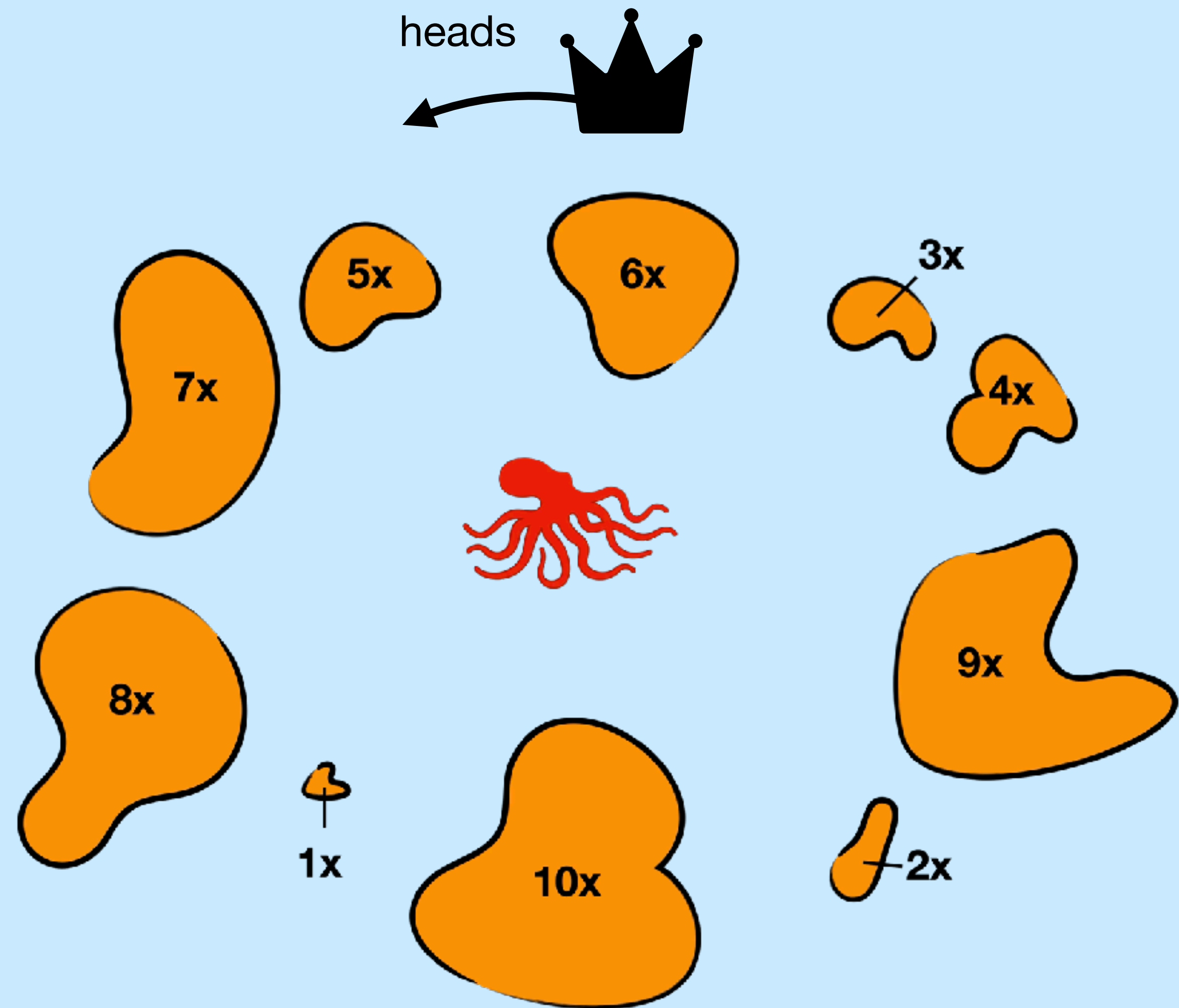
1. stay put or move to adjacent island by coin flip

2. if heads, considers clockwise movement, if tails, considers anti-clockwise

3. population size proposal
island = shells; population size
current island = stones

4. if 🐚 > 🪨 move to proposal island;
if 🪨 > 🐚 discard # 🪨 = 🐚
place remaining stones and shells in a bag and randomly pull one out

5. if 🐚 move to proposal island; if 🪨 stay.



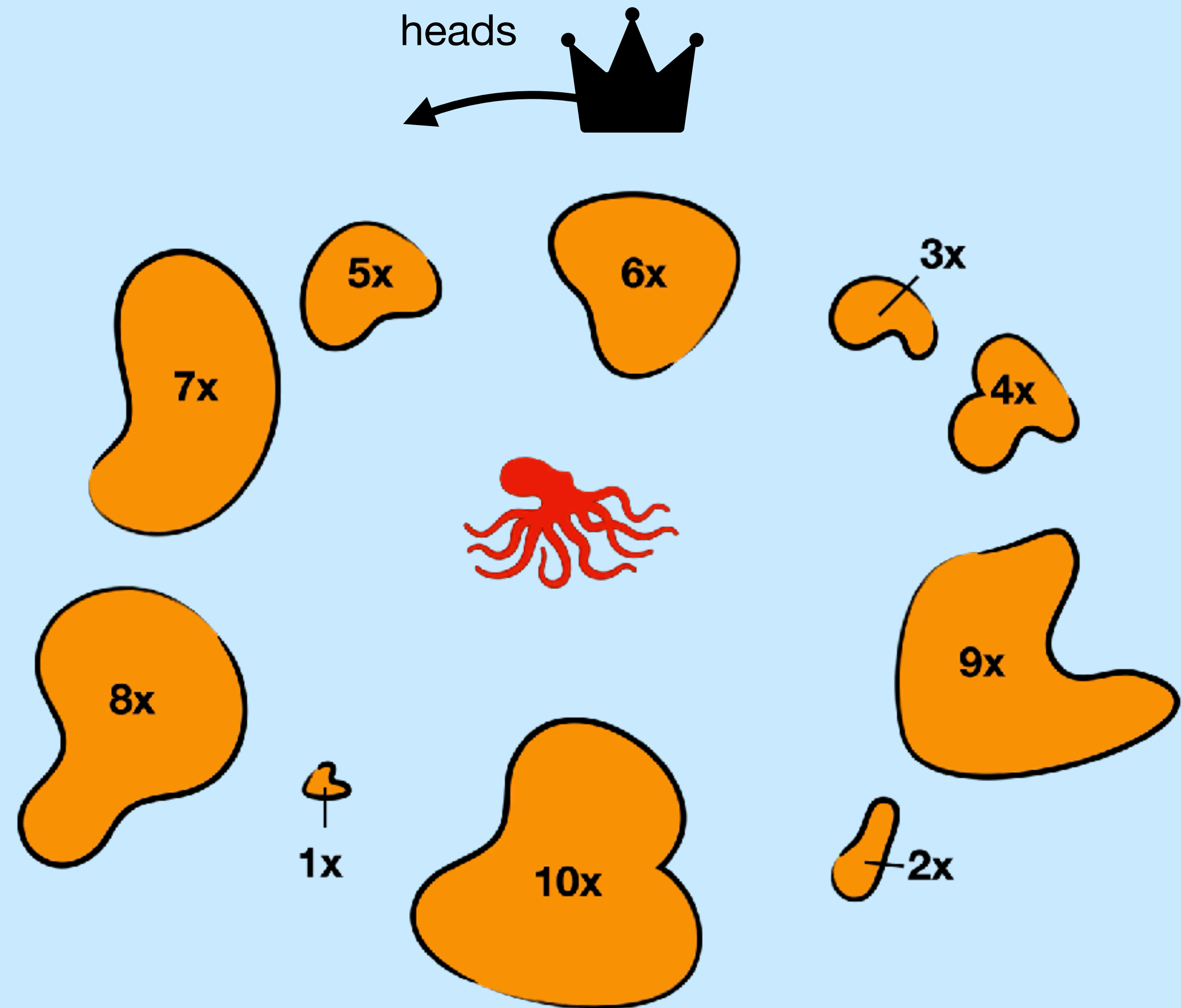
1. stay put or move to adjacent island by coin flip

2. if heads, considers clockwise movement, if tails, considers anti-clockwise

3. population size proposal
island = shells; population size
current island = stones

4. if 🐚 > 🪨 move to proposal island;
if 🪨 > 🐚 discard # 🪨 = 🐚
place remaining stones and shells in a bag and randomly pull one out

5. if 🐚 move to proposal island; if 🪨 stay.



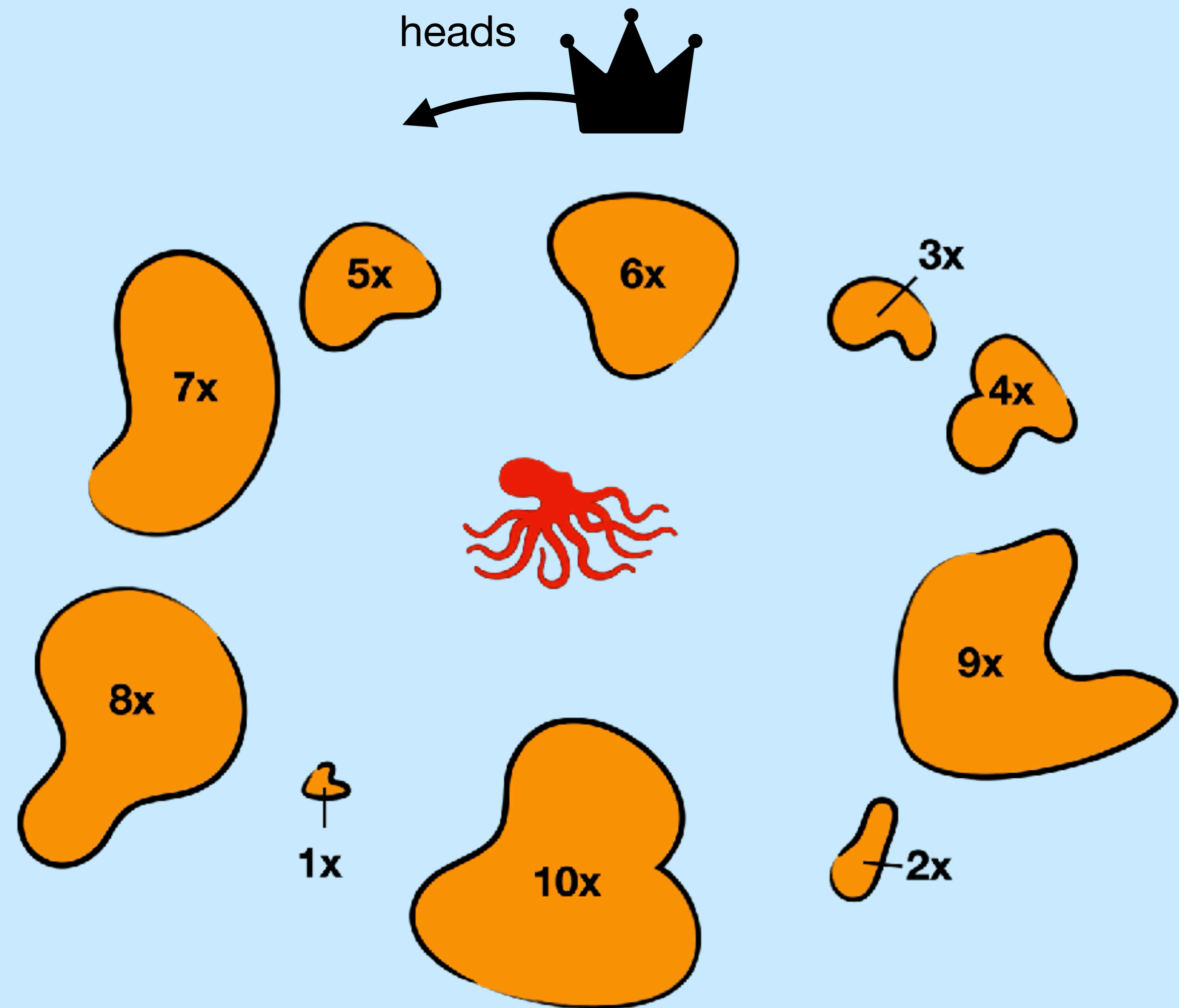
1. stay put or move to adjacent island by coin flip

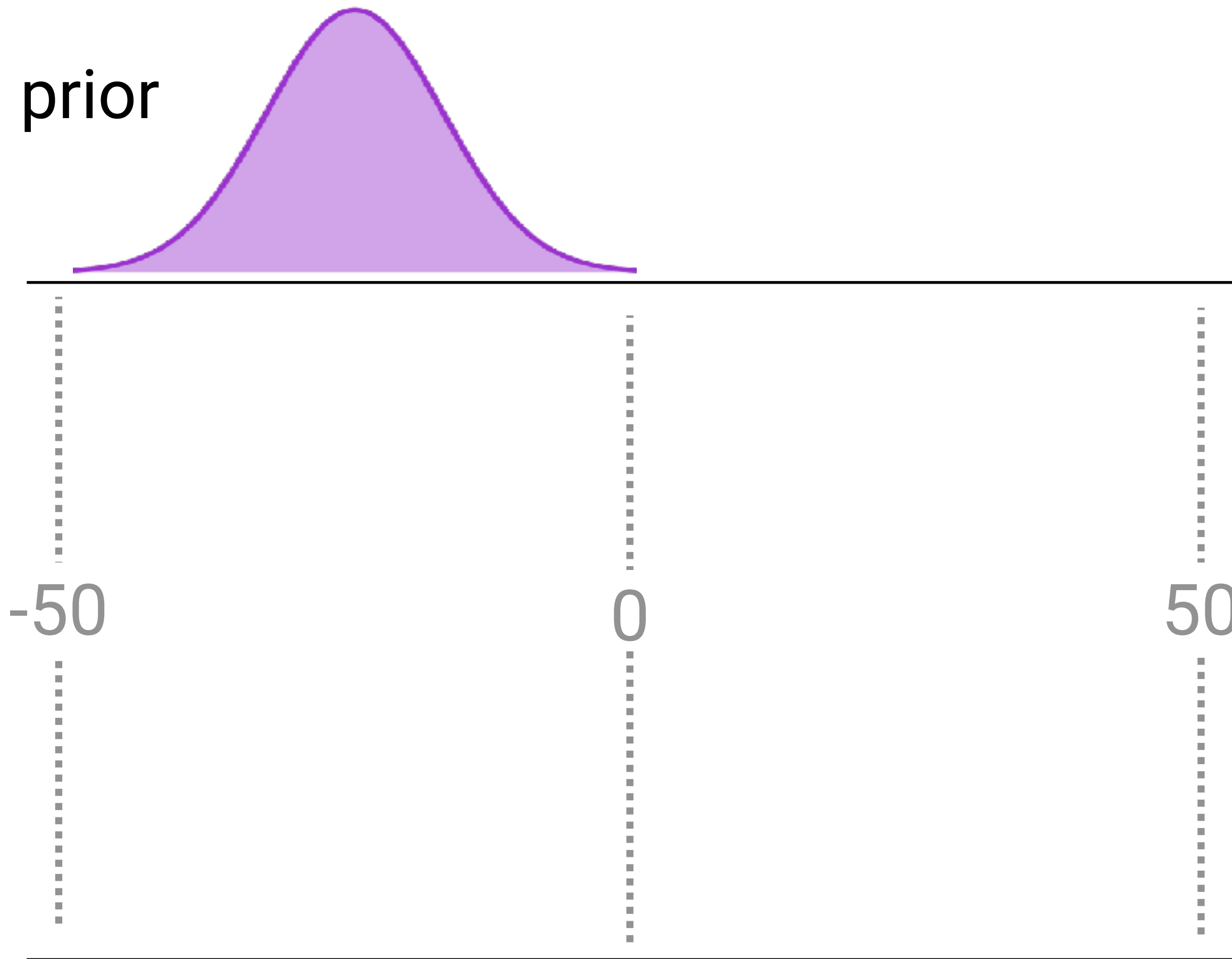
2. if heads, considers clockwise movement, if tails, considers anti-clockwise

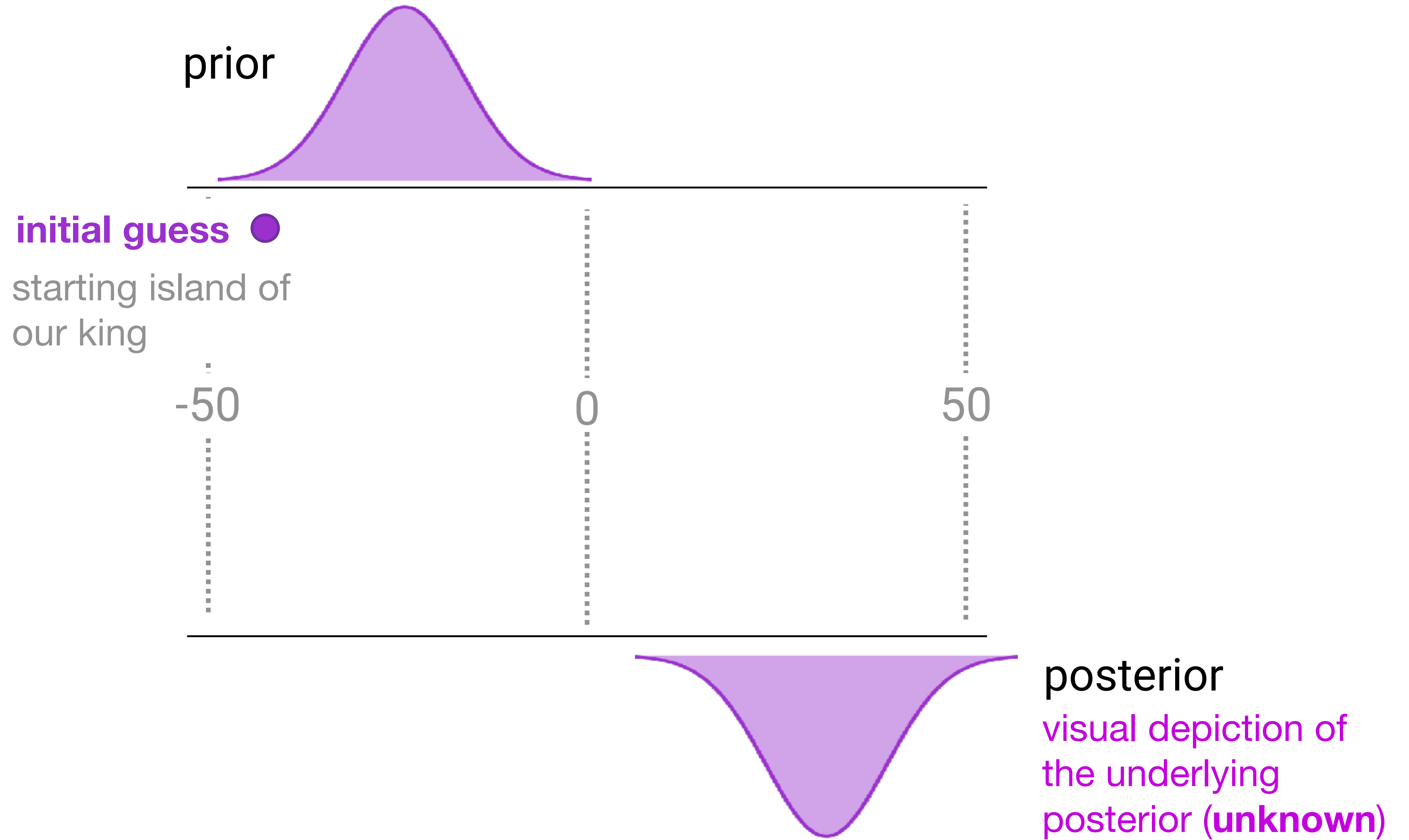
3. population size proposal
island = shells; population size current island = stones

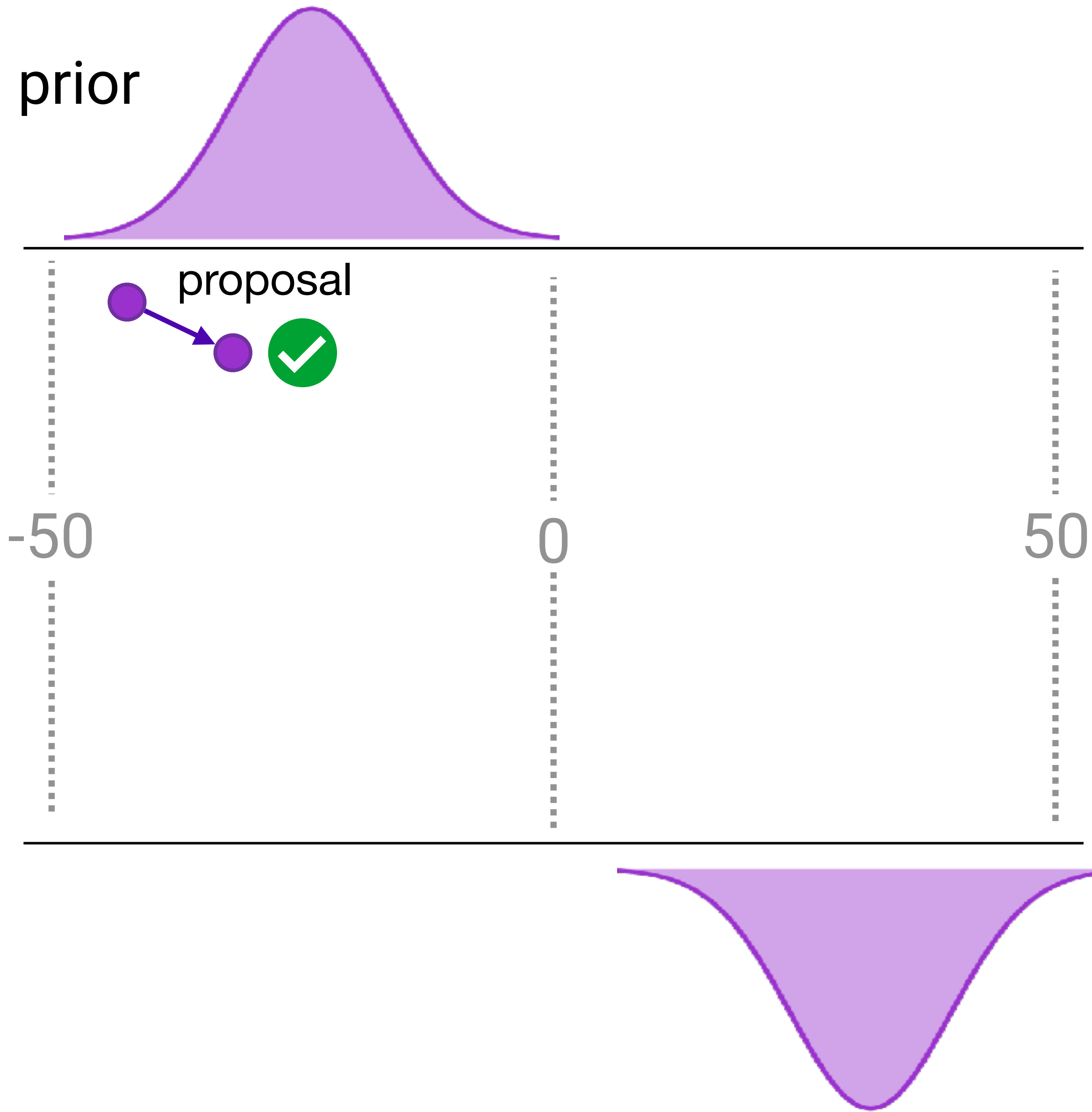
4. if 🐚 > 🪨 move to proposal island;
if 🪨 > 🐚 discard # 🪨 = 🐚
place remaining stones and shells in a bag and randomly pull one out

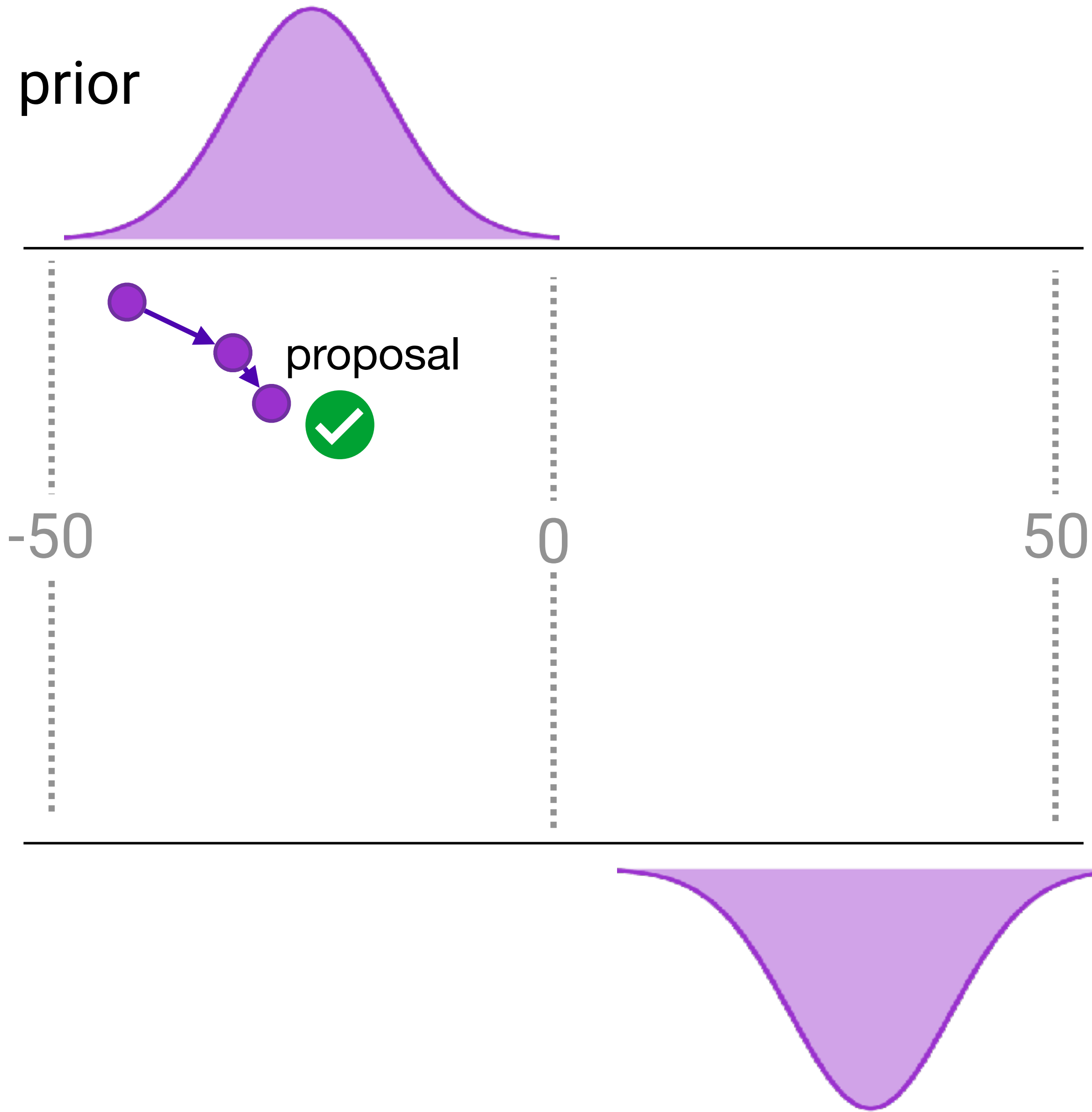
5. if 🐚 move to proposal island; if 🪨 stay.

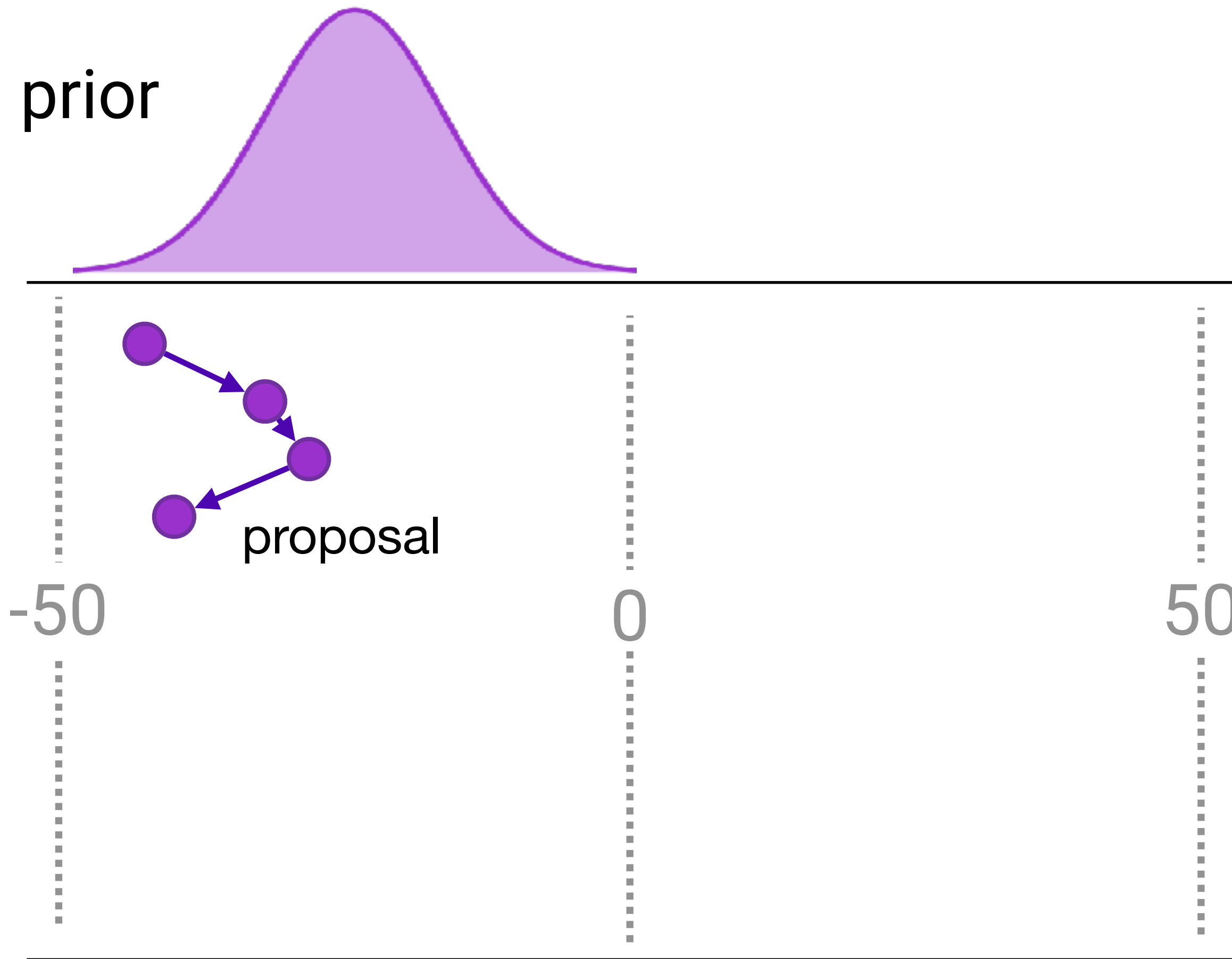




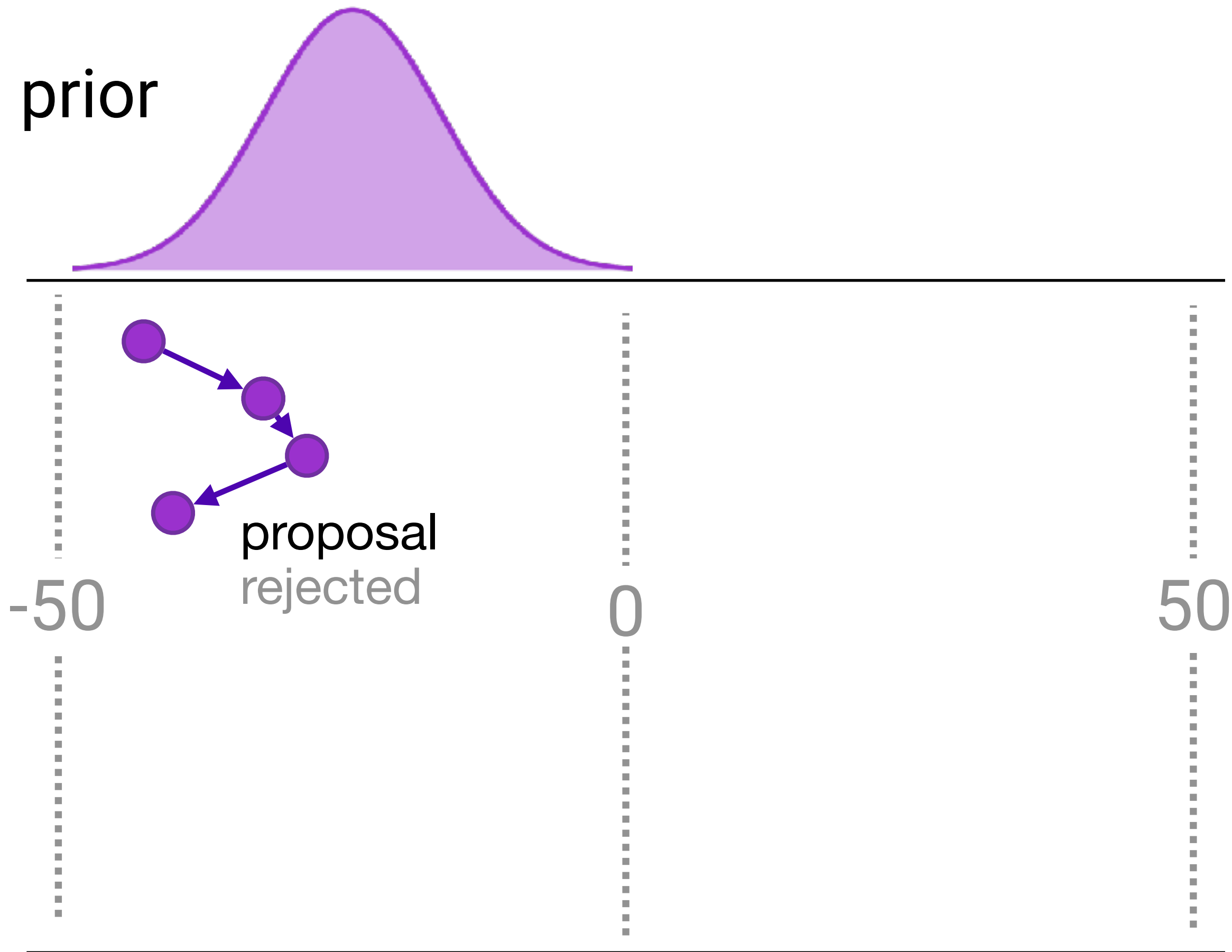




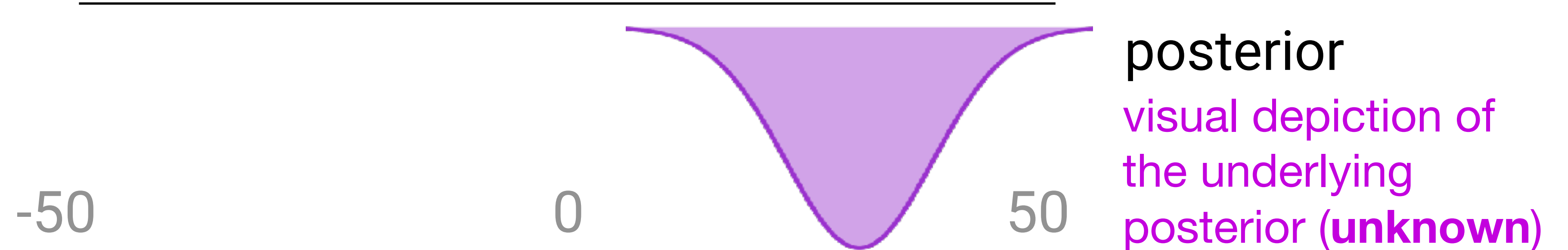
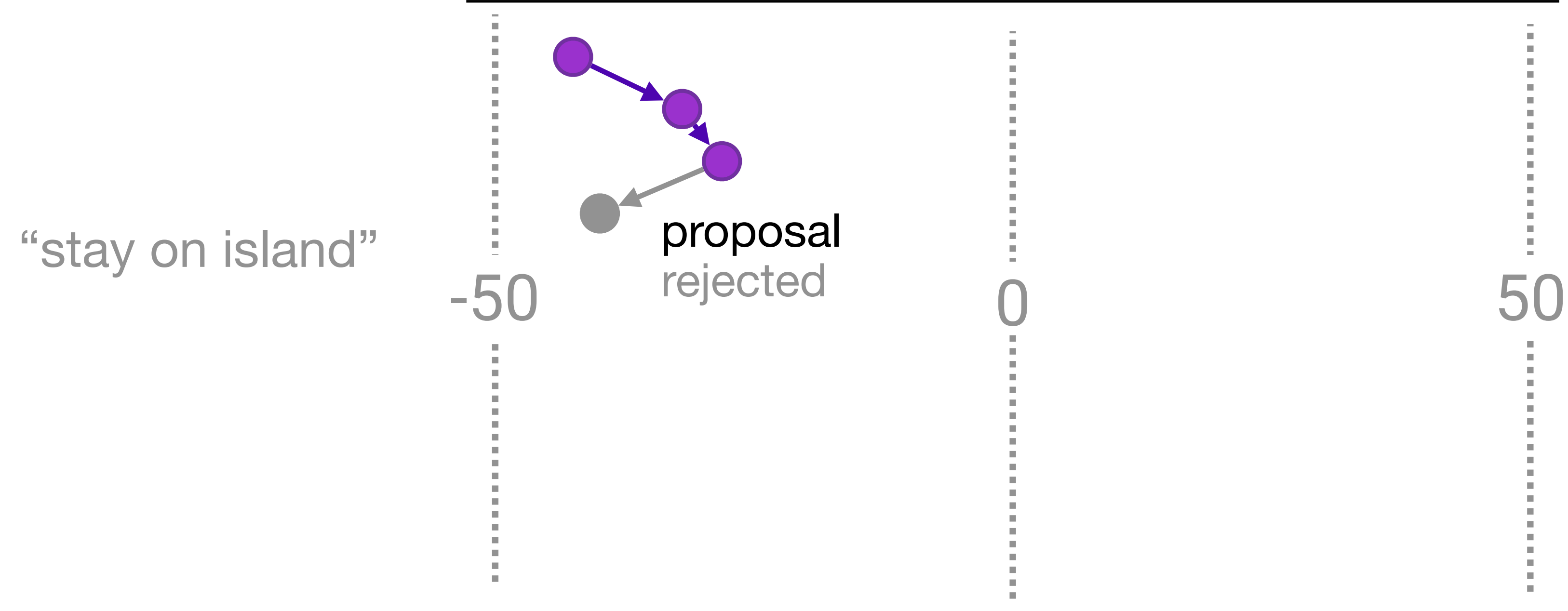
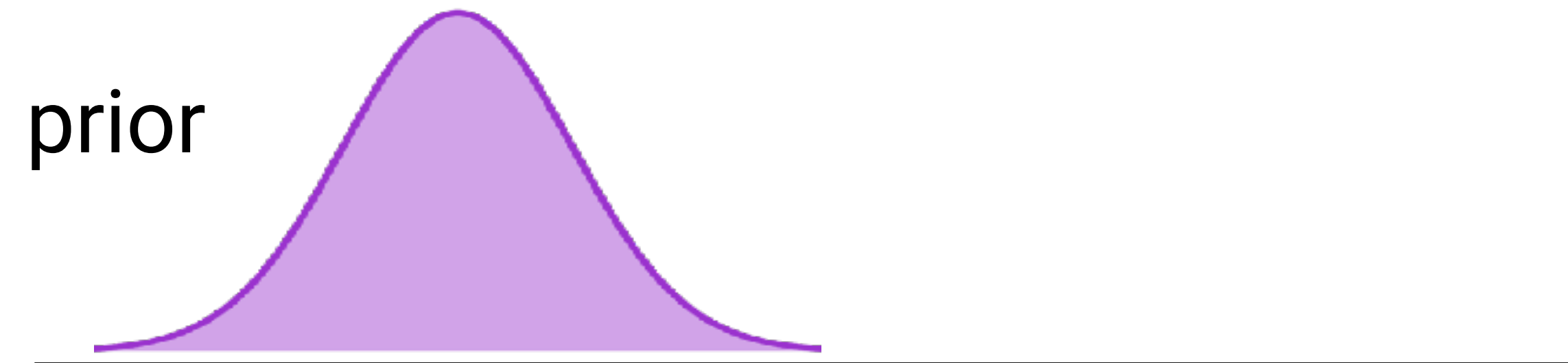


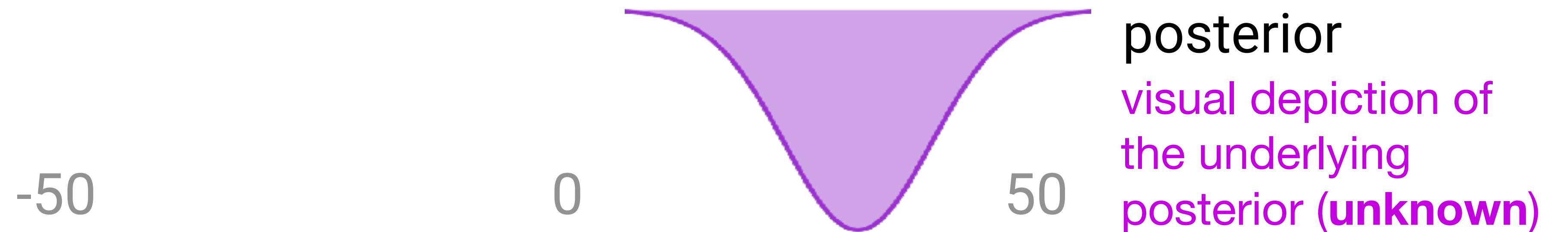
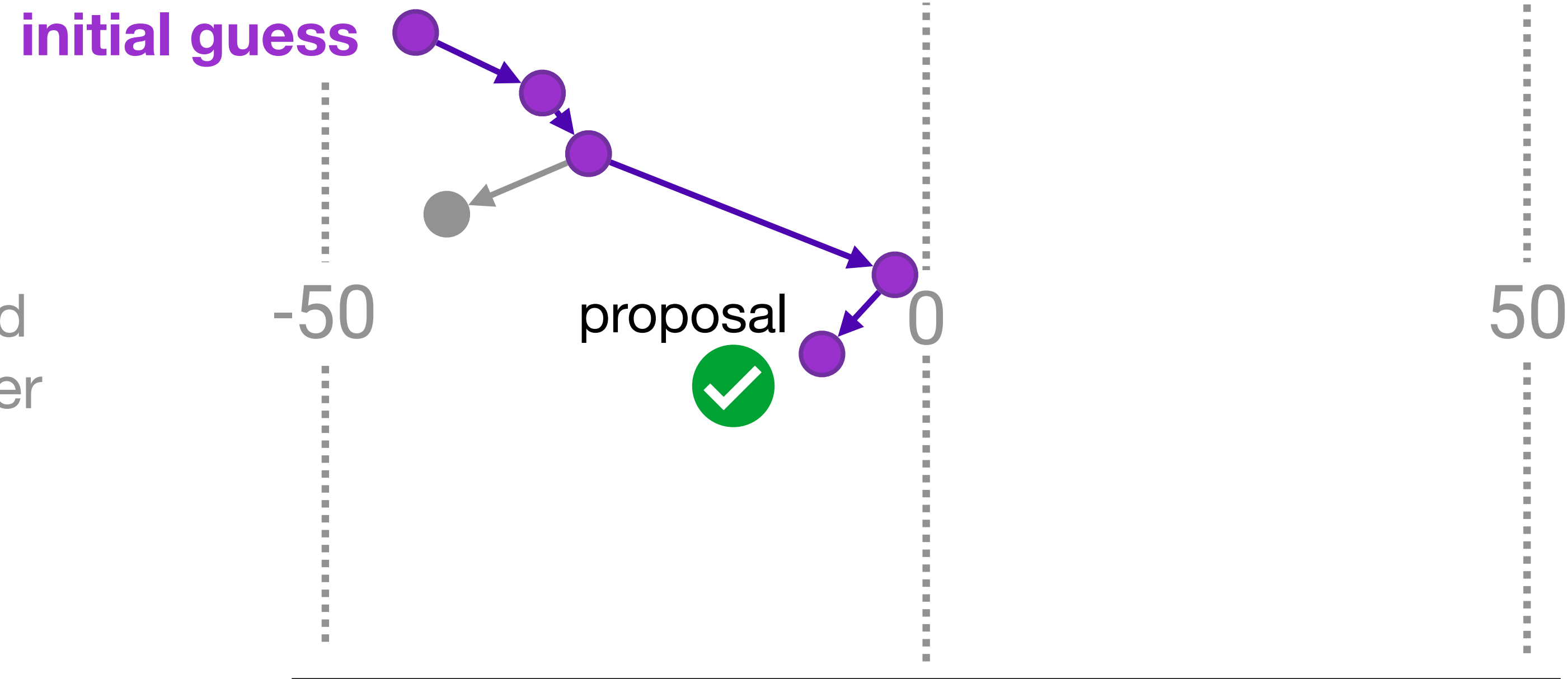
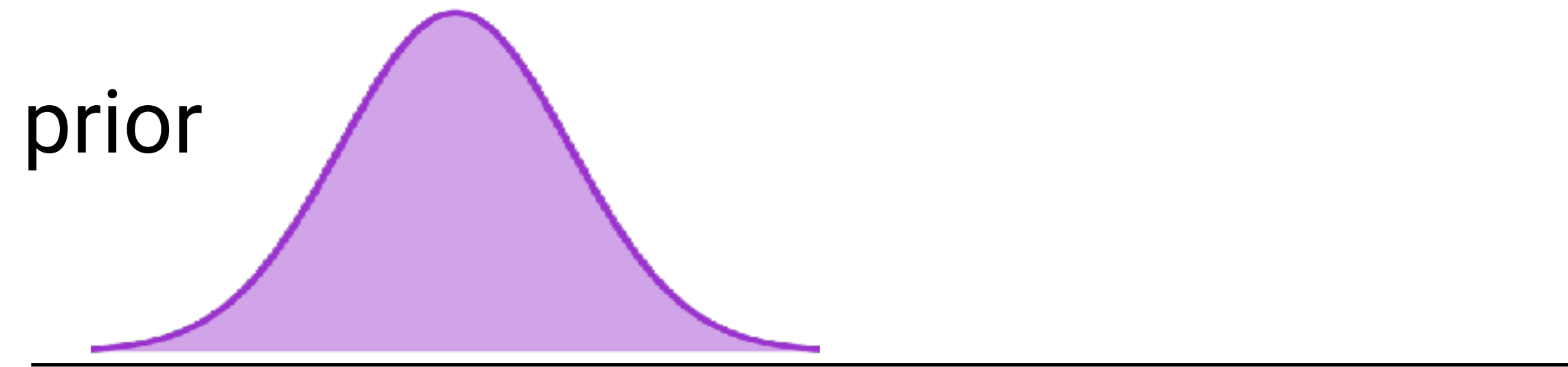


posterior
visual depiction of
the underlying
posterior (**unknown**)

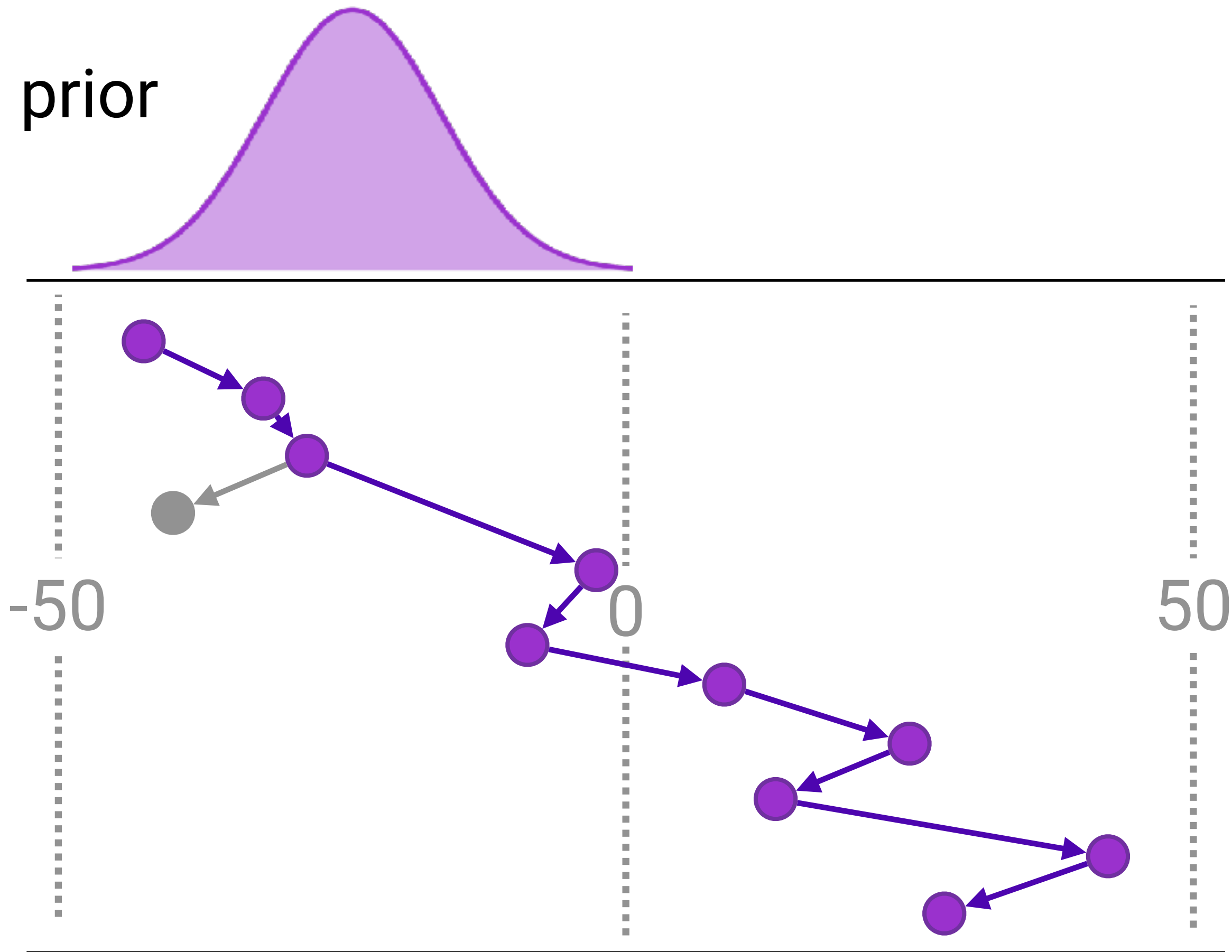


posterior
visual depiction of
the underlying
posterior (**unknown**)





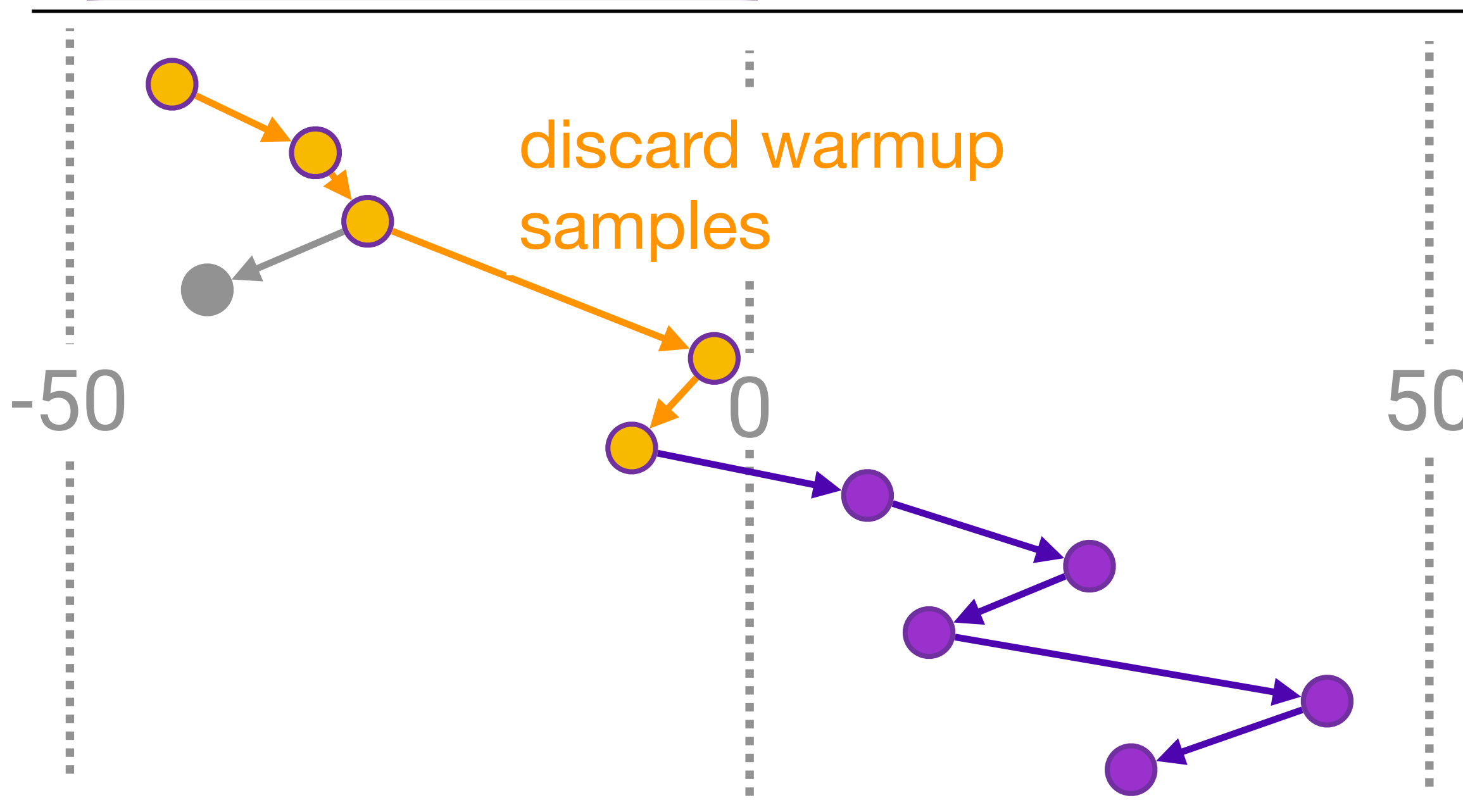
prior



posterior

visual depiction of
the underlying
posterior (**unknown**)

prior



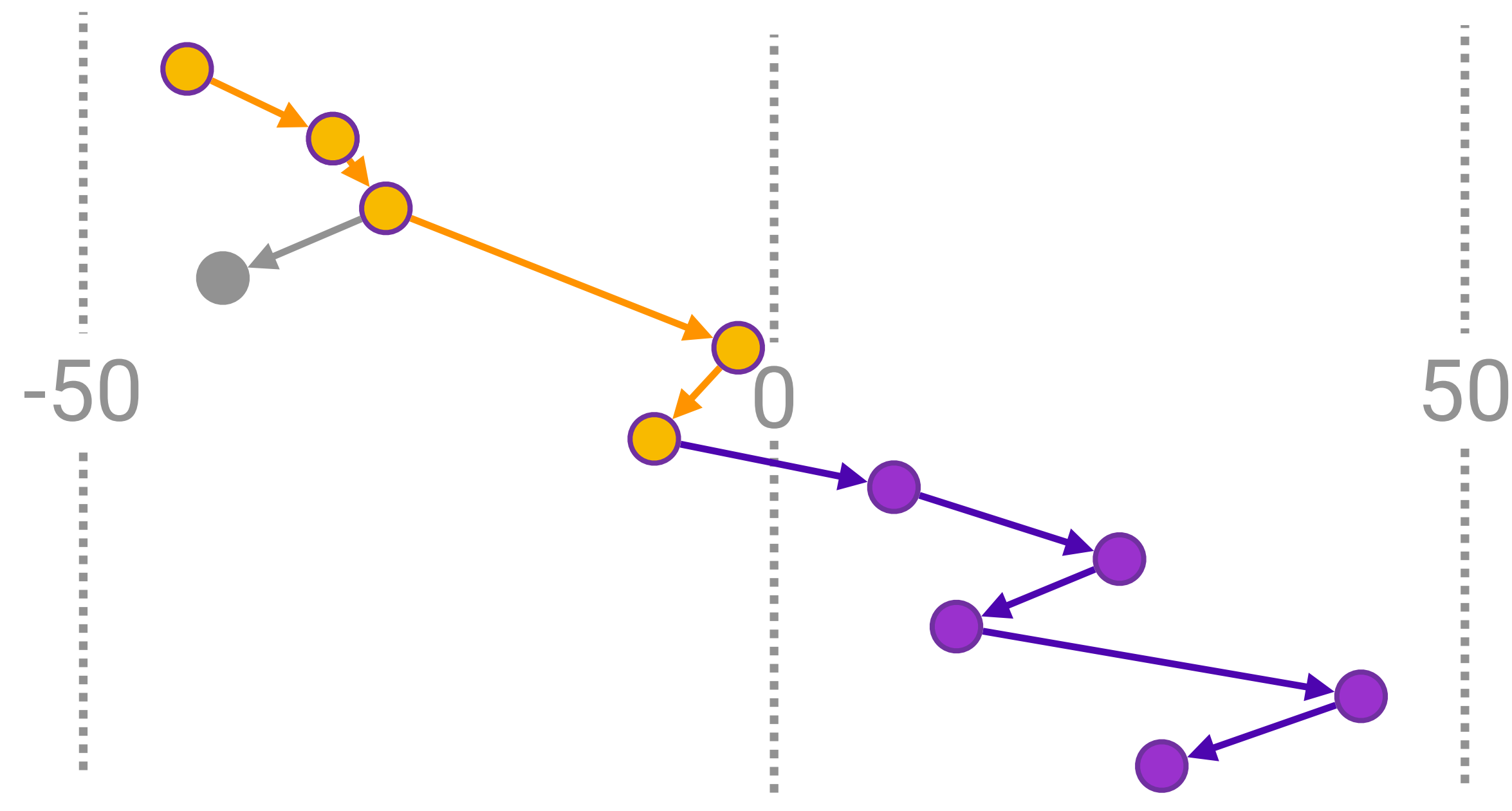
discard warmup
samples

keep post-warmup
samples (used for
inference)

posterior



prior



posterior



chain 1

prior

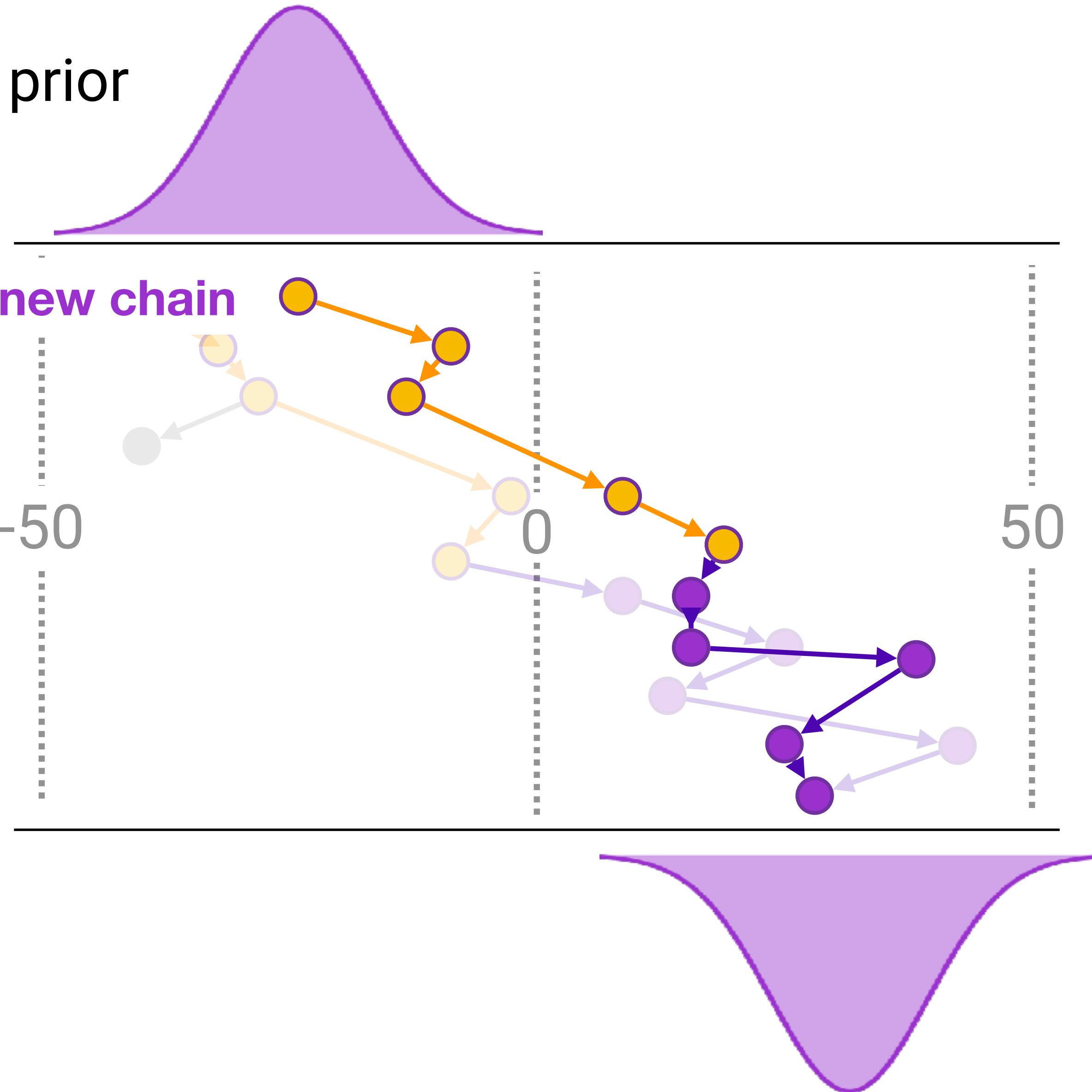
start new chain

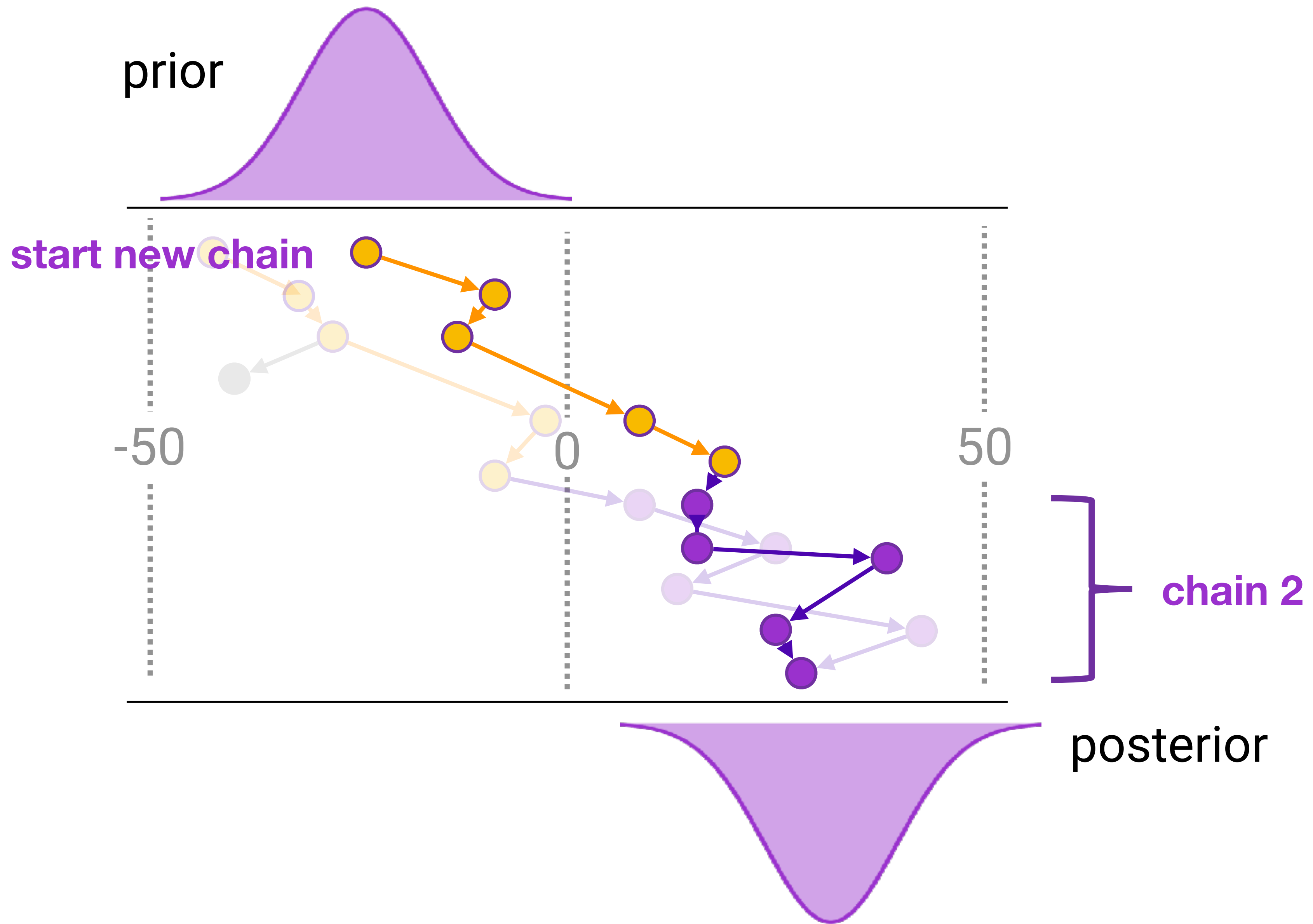
-50

0

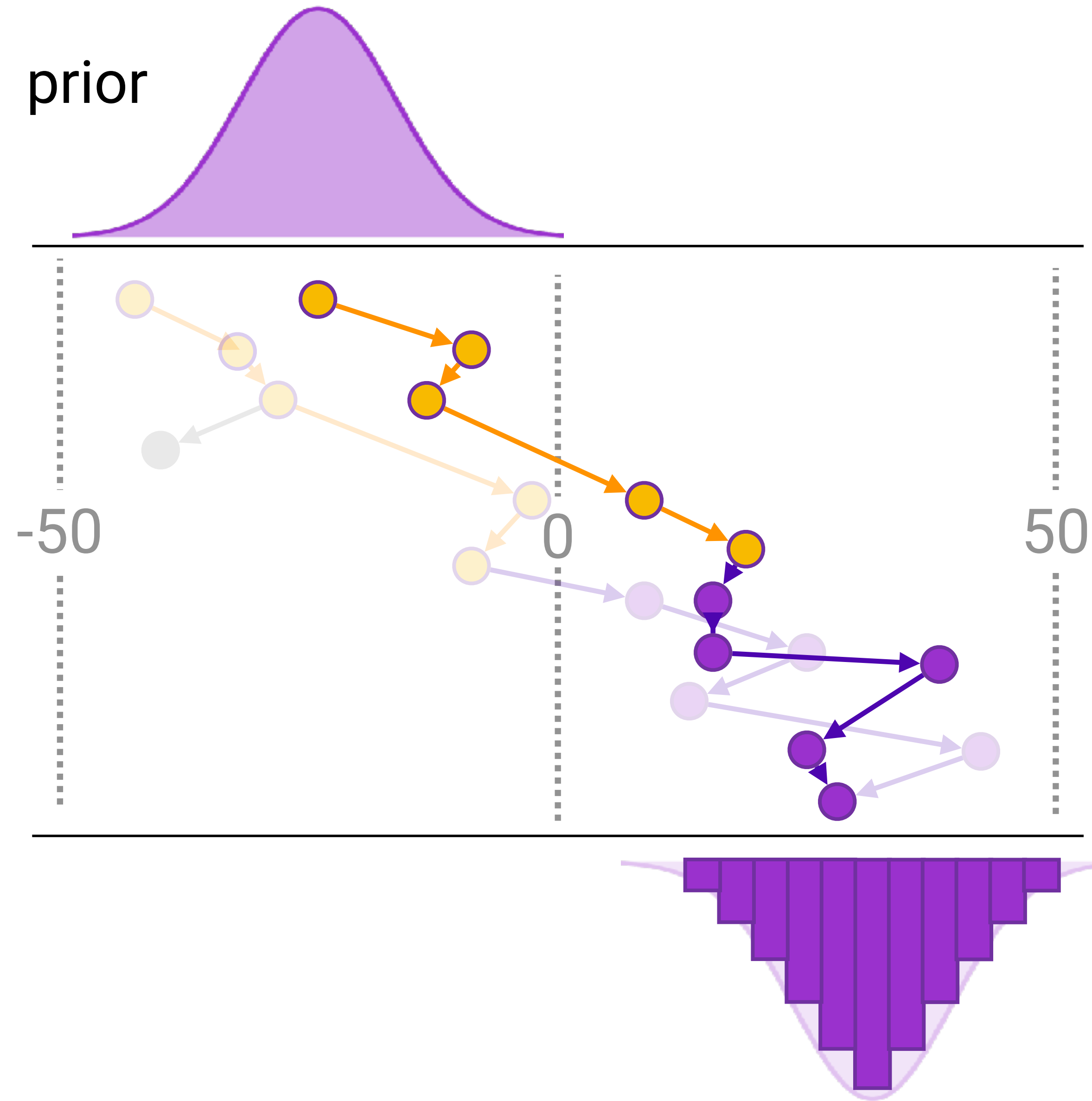
50

posterior





prior



posterior samples from
all chains approximate
the posterior

Divergent transitions

Warning message:

There were 17 divergent transitions after warmup. Increasing adapt_delta above 0.80 may help. See <http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup>

A warning that the trajectory may be unreliable because of bad geometry of the posterior

Suggests that there are aspects of the posterior that may not have been explored

Specify priors

```
my_prior <- c(prior(normal(110, 10), class = 'Intercept'),  
             prior(normal(0, 15), class = 'sigma'))
```

Fit model

```
normal_md1 <- brm(F0 ~ 1,  
                  family = gaussian(),  
                  prior = my_prior,  
                  data = pitch)
```

First part of output

```
Family: gaussian  
Links: mu = identity; sigma = identity  
Formula: F0 ~ 1  
Data: pitch (Number of observations: 32)  
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;  
       total post-warmup draws = 4000
```

```
my_prior <- c(prior(normal(110, 10), class = 'Intercept'),  
             prior(normal(0, 15), class = 'sigma'))
```

Fit model

```
normal_md1 <- brm(F0 ~ 1,  
                  family = gaussian(),  
                  prior = my_prior,  
                  data = pitch)
```

First part of output

```
Family: gaussian  
Links: mu = identity; sigma = identity  
Formula: F0 ~ 1  
Data: pitch (Number of observations: 32)
```

```
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;  
total post-warmup draws = 4000
```


Better priors than the uniform we did last time



```
my_prior <- c(prior(normal(110, 10), class = 'Intercept'),  
              prior(normal(0, 15), class = 'sigma'))
```

Fit model

```
normal_md1 <- brm(F0 ~ 1,  
                  family = gaussian(),  
                  prior = my_prior,  
                  data = pitch)
```

First part of output

```
Family: gaussian  
Links: mu = identity, sigma = identity  
Formula: F0 ~ 1  
Data: pitch (Number of observations: 32)
```

4x (2000 - 1000) = 4000

**Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
total post-warmup draws = 4000**


```
my_prior <- c(prior(normal(110, 10), class = 'Intercept'),  
             prior(normal(0, 15), class = 'sigma'))
```

Fit model

Thinning = keep every n th sample

```
normal_md1 <- brm(F0 ~ 1,  
                  family = gaussian(),  
                  prior = my_prior,  
                  data = pitch)
```

First part of output

```
Family: gaussian  
Links: mu = identity, sigma = identity  
Formula: F0 ~ 1  
Data: pitch (Number of observations: 32)
```

4x (2000 - 1000) = 4000

keep every n th sample

```
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;  
total post-warmup draws = 4000
```

```
my_prior <- c(prior(normal(110, 10), class = 'Intercept'),  
             prior(normal(0, 15), class = 'sigma'))
```

Fit model

```
normal_md1 <- brm(F0 ~ 1,  
                  family = gaussian(),  
                  prior = my_prior,  
                  data = pitch)
```

First part of output

```
Family: gaussian  
Links: mu = identity, sigma = identity  
Formula: F0 ~ 1  
Data: pitch (Number of observations: 32)
```

Draws: 4 chains, each with iter = 2000; warmup = 1000; **thin = 2**;
total post-warmup draws = 4000

Thinning = keep every *n*th sample

Reason #1: Memory constraints

Reason #2: Autocorrelation

But, you may be throwing away valid samples → keep default

$$4 \times (2000 - 1000) = 4000 / 2 = 2000$$

keep every *n*th sample

Specify priors

```
my_prior <- c(prior(normal(110, 10), class = 'Intercept'),  
             prior(normal(0, 15), class = 'sigma'))
```

Fit model

```
normal_md1 <- brm(F0 ~ 1,  
                  family = gaussian(),  
                  prior = my_prior,  
                  data = pitch,  
  
                  # MCMC specs:  
  
                  chains = 4, cores = 4,  
                  iter = 2000, warmup = 1000,  
                  thin = 1, seed = 666)
```


Specify priors

```
my_prior <- c(prior(normal(110, 10), class = 'Intercept'),  
             prior(normal(0, 15), class = 'sigma'))
```

Fit model

```
normal_md1 <- brm(F0 ~ 1,  
                  family = gaussian(),  
                  prior = my_prior,  
                  data = pitch,  
  
                  # MCMC specs:  
  
                  chains = 4, cores = 4,  
                  iter = 2000, warmup = 1000,  
                  thin = 1, seed = 666)
```

runs each chain in parallel
using a different processor
("core") of your computer
(modern computers are almost
always multicore)

Specify priors

```
my_prior <- c(prior(normal(110, 10), class = 'Intercept'),  
             prior(normal(0, 15), class = 'sigma'))
```

Fit model

```
normal_md1 <- brm(F0 ~ 1,  
                  family = gaussian(),  
                  prior = my_prior,  
                  data = pitch,  
  
                  # MCMC specs:  
  
                  chains = 4, cores = 4,  
                  iter = 2000, warmup = 1000,  
                  thin = 1, seed = 666)
```

set random seed to any number of your liking, and each time you'll get the same results

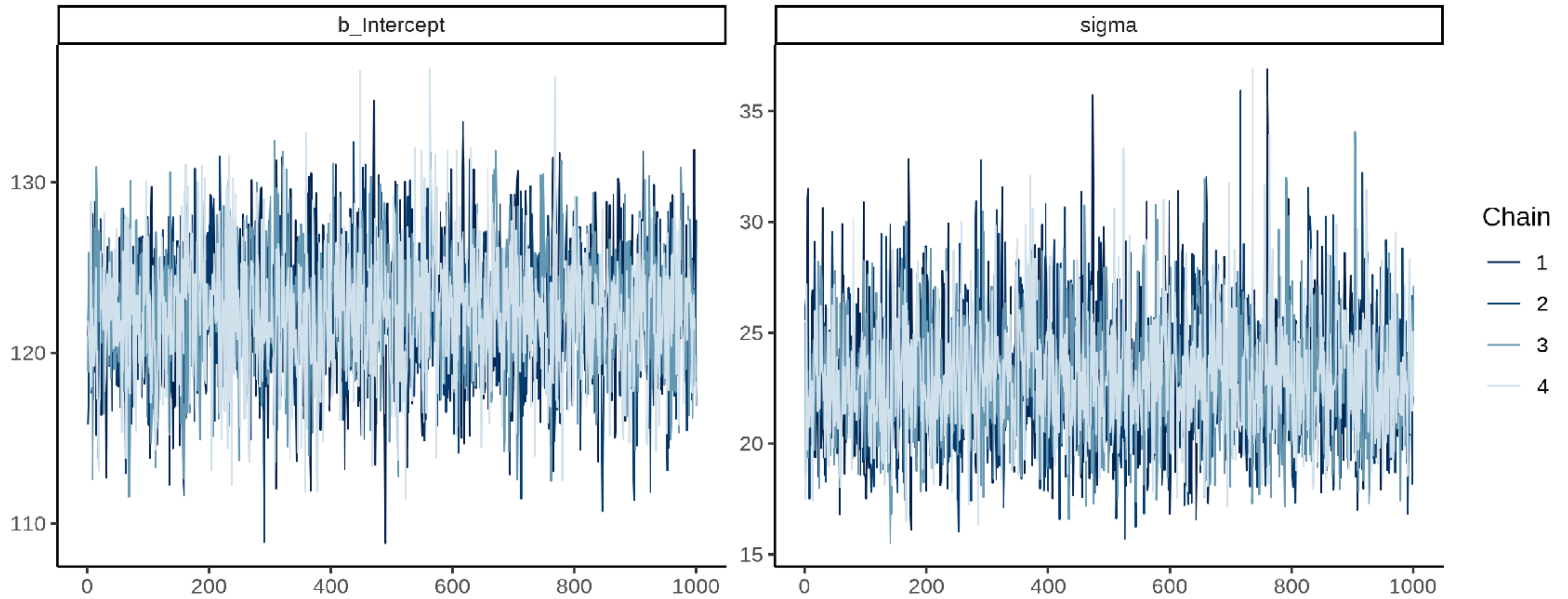
Convergence diagnostics

aka how to know if we can
trust the results?



Convergence diagnostics

```
mcmc_plot(normal_md1, type = 'trace')
```

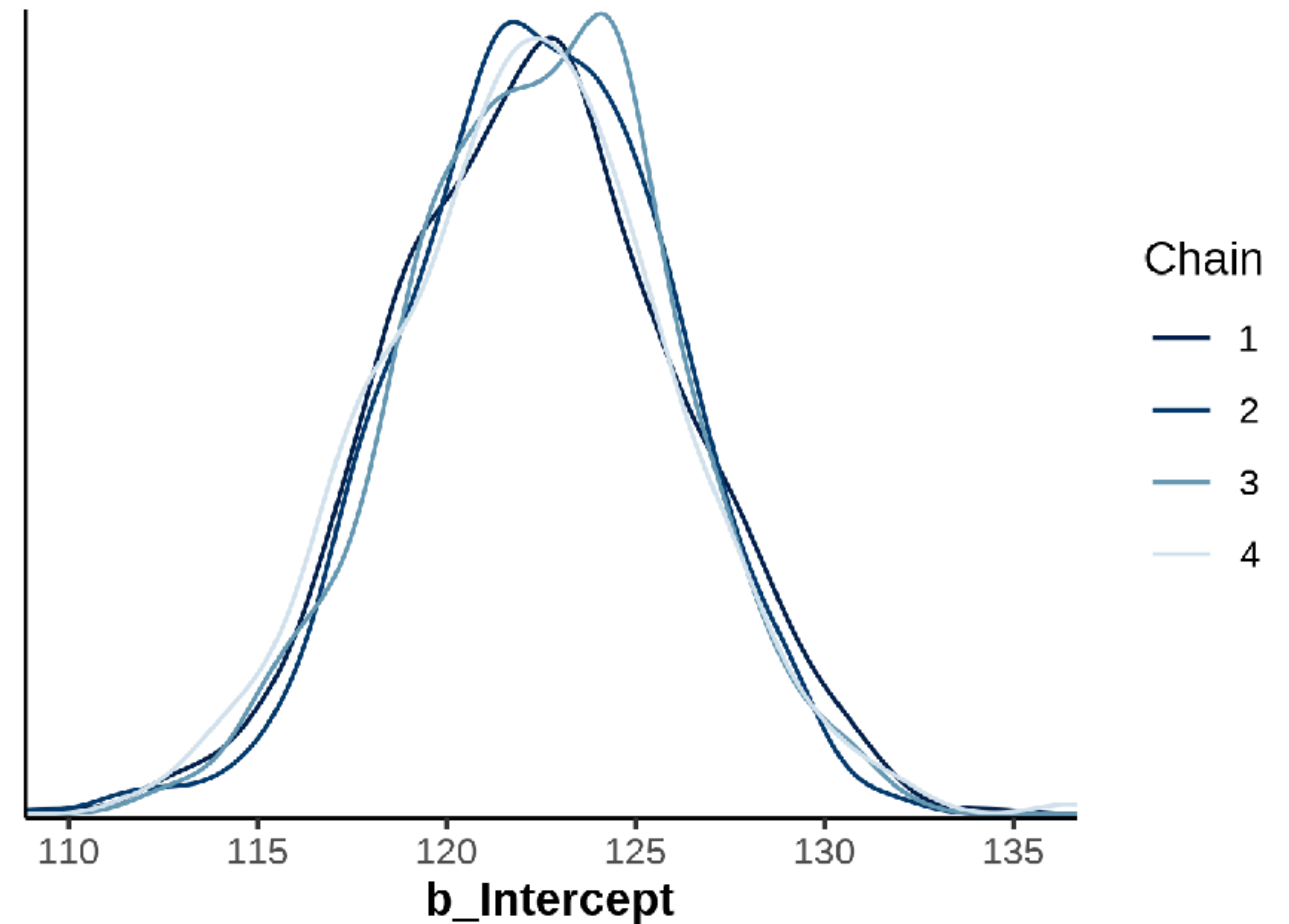
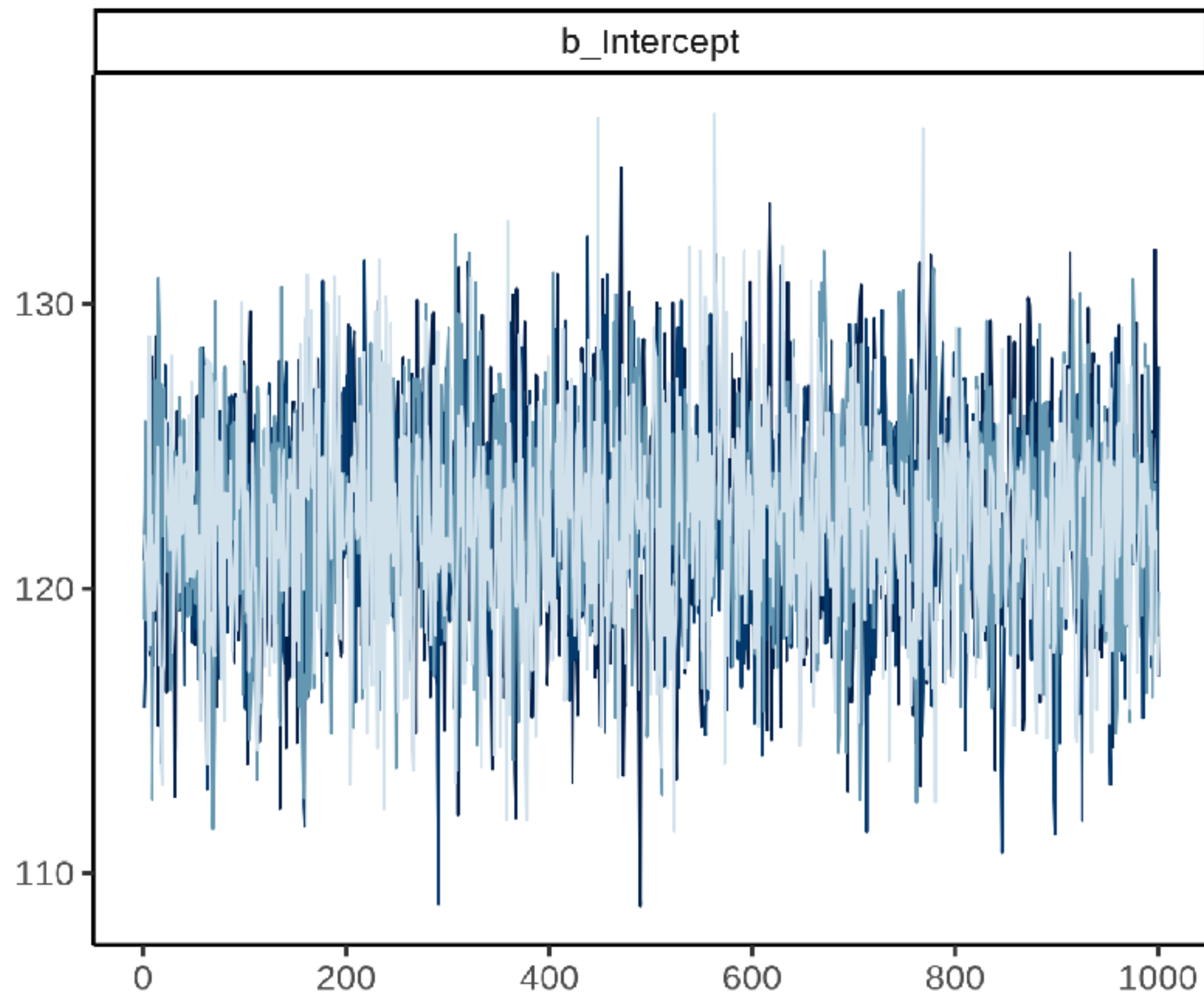


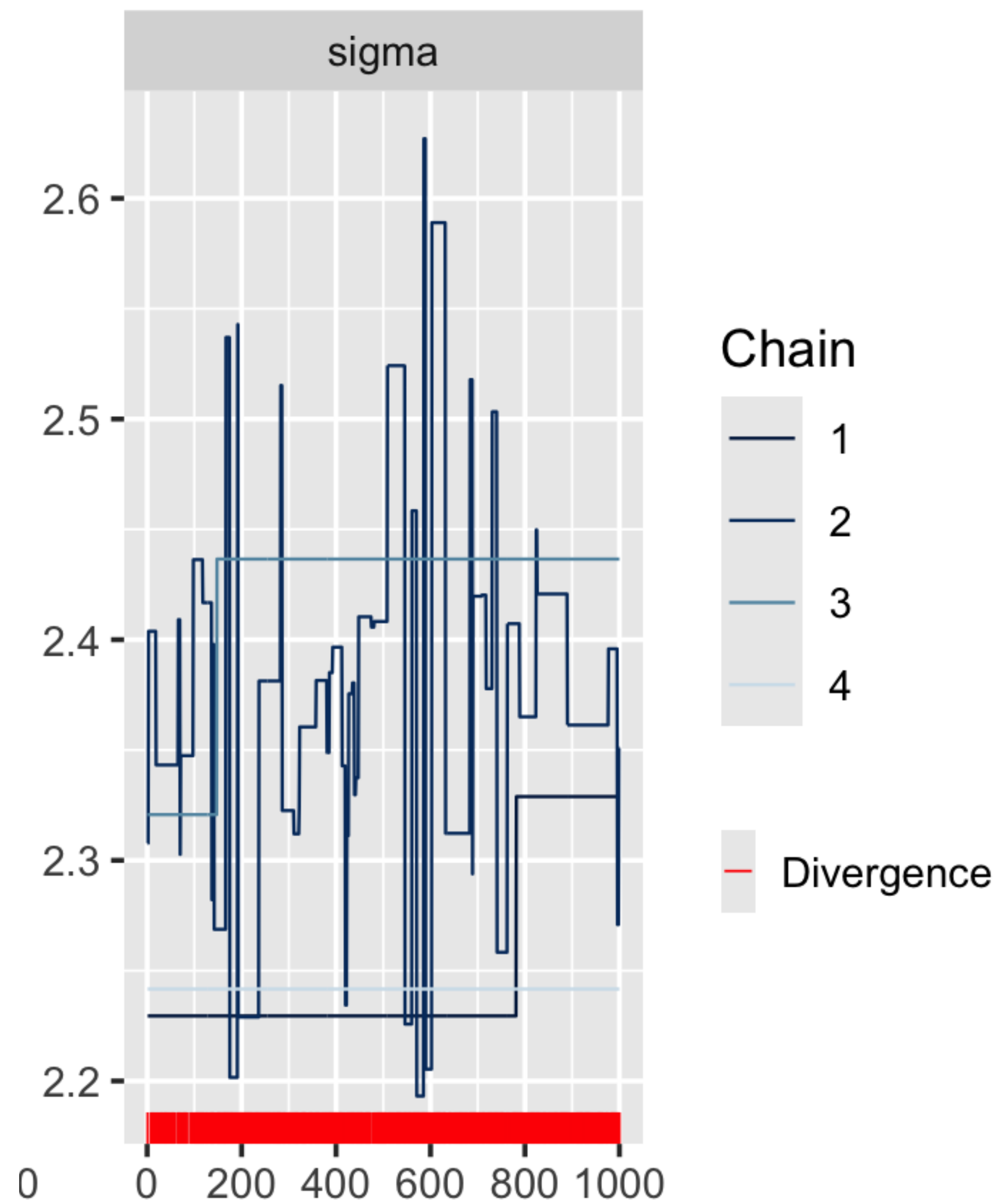
1000 post-warmup iterations

Convergence diagnostics

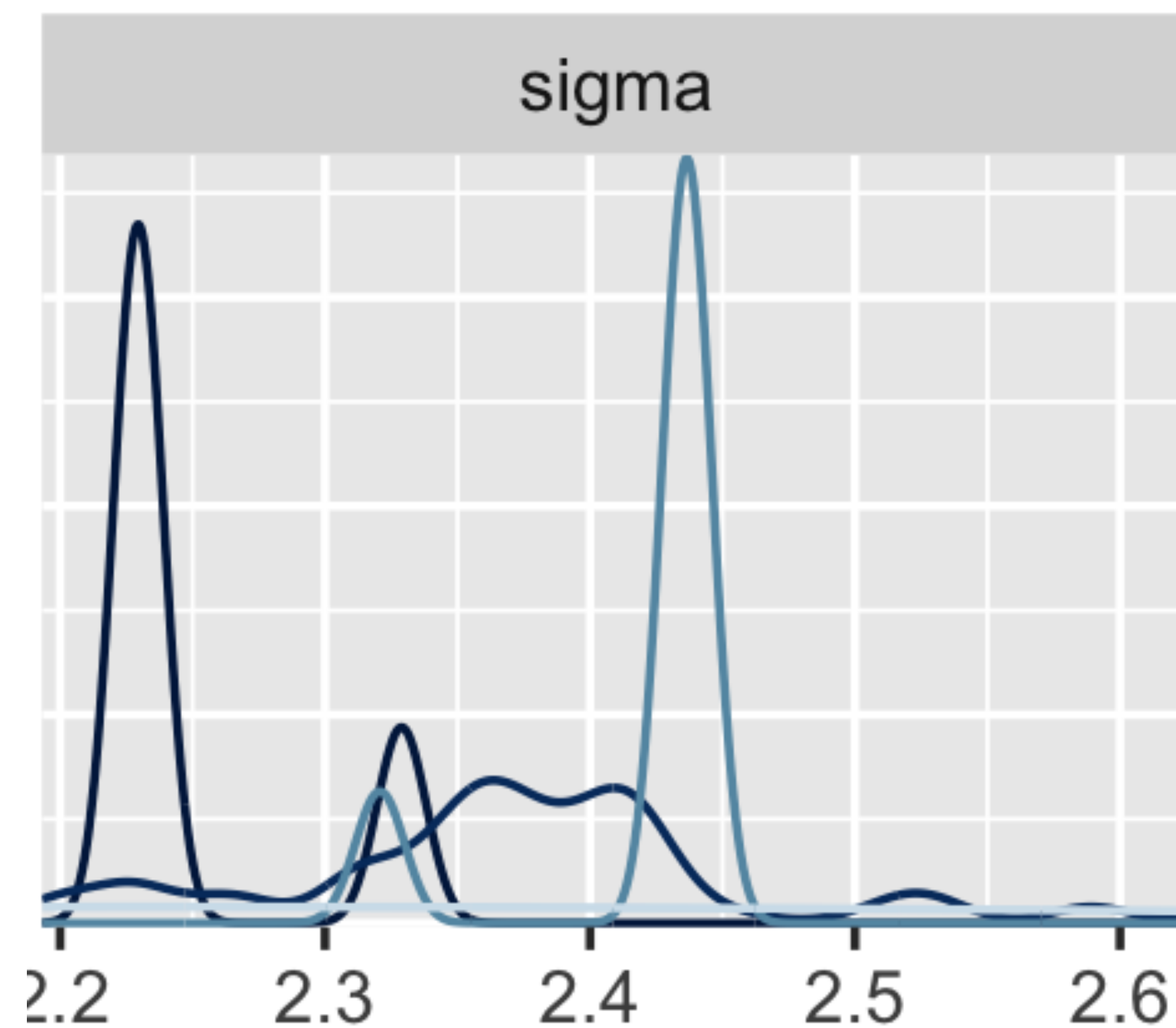
```
mcmc_plot(normal_md1, type = 'trace')
```

```
mcmc_dens_overlay(uniform_prior_md12, pars = 'b_Intercept')
```





Example of a bad model



Convergence diagnostics

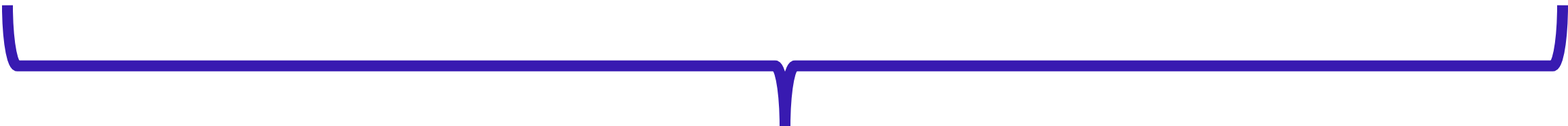
Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	122.34	3.69	115.03	129.47	1.00	$\overline{2922}$	$\overline{2786}$

Convergence diagnostics

Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk ESS	Tail ESS
Intercept	122.34	3.69	115.03	129.47	1.00	$\overline{2922}$	$\overline{2786}$

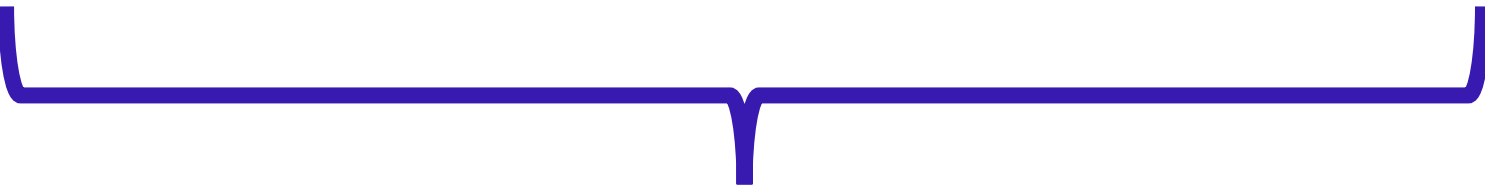


information about the posterior

Convergence diagnostics

Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	122.34	3.69	115.03	129.47	1.00	$\overline{2922}$	$\overline{2786}$



information about
MCMC estimation

Convergence diagnostics

Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk ESS	Tail ESS
Intercept	122.34	3.69	115.03	129.47	1.00	2922	2786

$$\hat{R} = \sqrt{\frac{\text{between chain variance}}{\text{within chain variance}}}$$

1.0 = chains have converged

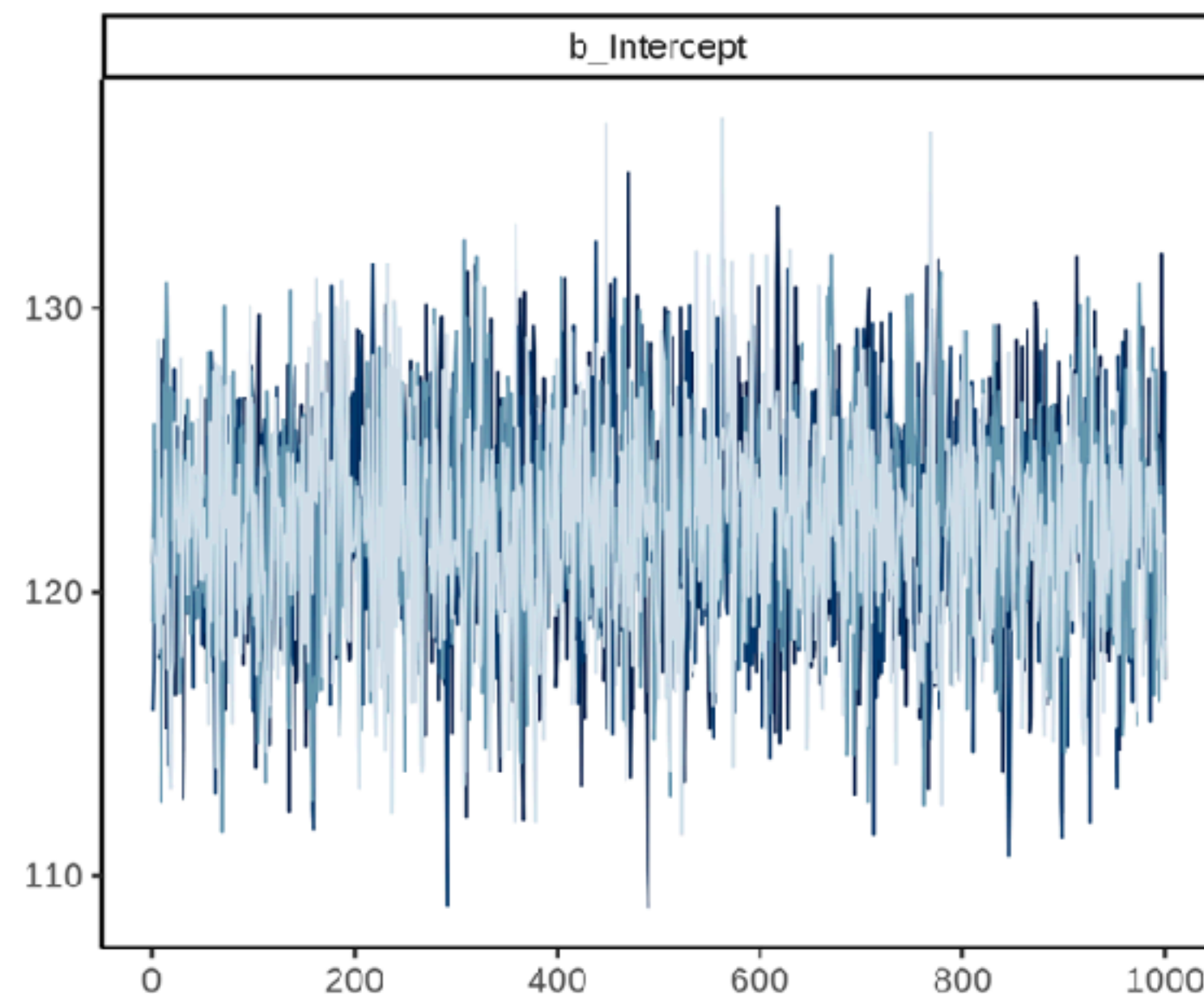
>1.01 = potential lack of convergence

>>1 = very poor mixing

Convergence diagnostics

Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk ESS	Tail ESS
Intercept	122.34	3.69	115.03	129.47	1.00	2922	2786



$$\hat{R} = \sqrt{\frac{\text{between chain variance}}{\text{within chain variance}}}$$

- ➡ **1.0** = chains have converged
- >1.01** = potential lack of convergence
- >>1** = very poor mixing

Convergence diagnostics

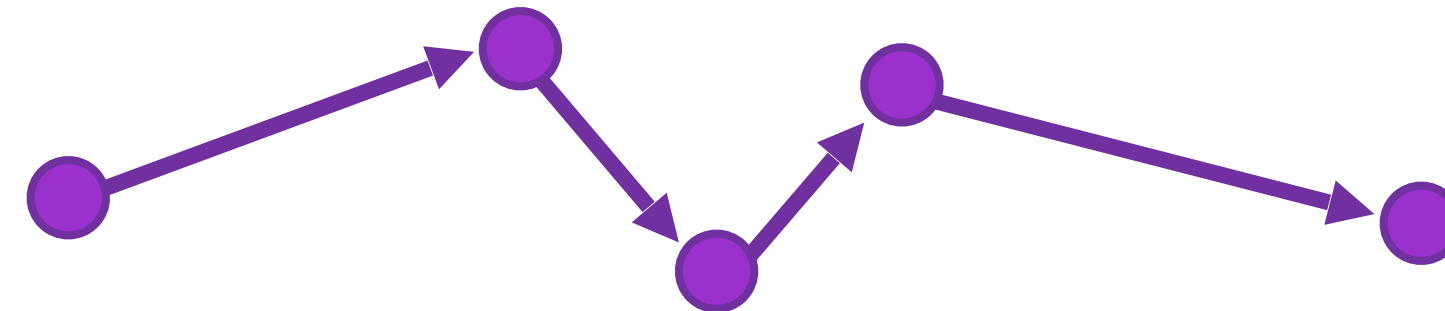
Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	122.34	3.69	115.03	129.47	1.00	$\overline{2922}$	$\overline{2786}$

Convergence diagnostics

Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk ESS	Tail ESS
Intercept	122.34	3.69	115.03	129.47	1.00	$\overline{2922}$	$\overline{2786}$

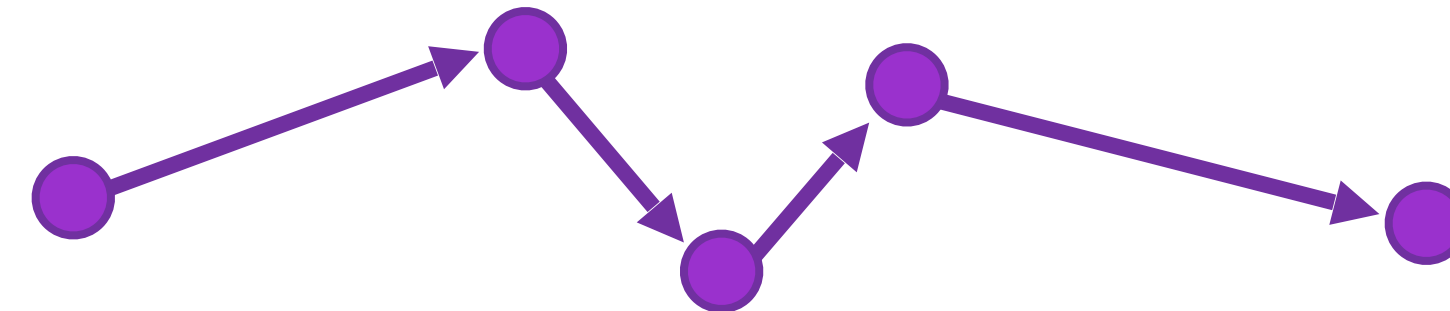


draws are not independent,
and can be autocorrelated

Convergence diagnostics

Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk ESS	Tail ESS
Intercept	122.34	3.69	115.03	129.47	1.00	$\overline{2922}$	$\overline{2786}$



draws are not independent,
and can be autocorrelated

strong autocorrelation =
4,000 samples don't give you
4,000 independent pieces of
information

“effective sample size” (ESS) =
this metric estimates the number
of independent samples your
autocorrelated MCMC samples
are equivalent to

Convergence diagnostics

Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk ESS	Tail ESS
Intercept	122.34	3.69	115.03	129.47	1.00	$\overline{2922}$	$\overline{2786}$

reflects typical
draws

Convergence diagnostics

focuses on
extreme
quantiles

Model output

	Estimate	Est.Error	1-95% CI	u-95% CI	Rhat	Bulk ESS	Tail ESS
Intercept	122.34	3.69	115.03	129.47	1.00	$\overline{2922}$	$\overline{2786}$

reflects typical
draws

Rules of thumb for ESS

- **higher ESS = better** (more independent samples)
- **bulk ESS > 400** = usually 'safe' for mean/variance estimates
- **tail ESS > 100** = usually 'safe' for credible intervals

Problem

divergent transitions > 0

Rhat > 1.0

Bulk ESS < 400

Tail ESS < 100

Solution

Increase warmup period

```
brm(...,  
  
    # MCMC specs:  
  
    iter = 6000, warmup = 4000)
```

Increase adapt_delta

```
brm(...,  
  
    # MCMC specs:  
  
    control =  
      list(adapt_delta = 0.99))
```


Solution

make sure to increase **iter**
accordingly so you have enough
post-warmup samples

Increase warmup period

```
brm( . . . ,  
      # MCMC specs:  
      iter = 6000, warmup = 4000 )
```



Increase adapt_delta

```
brm( . . . ,  
      # MCMC specs:  
      control =  
        list(adapt_delta = 0.99) )
```

default is 0.8; increasing it to 0.9, 0.95
or 0.99 means that HMC takes
smaller, more careful steps
(will slow the fitting process)



There were 5 transitions after warmup that exceeded the maximum treedepth. Increase max_treedepth above 12

Solution

Increase max_treedepth

```
brm(...,  
      # MCMC specs:  
  
      control =  
        list(max_treedepth = 13))
```



Frequentist estimation (lme4)

Find the single “**best fit**” set of parameters / **point estimates** that make the data most likely (MLE, maximum likelihood estimation)

Optimization algorithms to climb to the highest point of the likelihood “hill”

Usually faster

Bayesian estimation (brms/Stan)

Explore the **full range of parameter values** that are plausible, given model, data, and priors

Use random walks (chains of draws) to wander around **the entire posterior “landscape”**

Much slower